QUANTUM COMPUTATION NOTES

The Essence of Quantum Computing

Foundations, Algorithms, and Applications

Compiled by

Tanishk Modi

Naval Air Warfare Center Aircraft Division

Abstract

Below are the notes I have compiled over the time of my internship at NAWCAD, under the mentorship of Dr. YikLoon Lee.

Contents

1 Motivation **3**
1.1 Bits . . . 3
1.2 Limitations of Classical Computers . . . 3
1.3 The Quantum Promise . . . 3

2 The Qubit: Mathematical and Geometric Foundations **4**
2.1 Superposition . . . 4
2.2 Probabilities and Amplitudes . . . 4
2.3 Measuring a Qubit . . . 5
2.4 The Scaling Table: Classical vs. Quantum . . . 5
2.5 Deriving the Bloch Sphere (From Cartesian to Spherical) . . . 6
2.6 The Bloch Sphere . . . 7
2.7 Circle Notation . . . 9

3 Single-Qubit Quantum Gates **10**
3.1 What is a Quantum Gate? . . . 10
3.2 Why Unitary? . . . 10
3.3 Visualizing Unitary Operations . . . 11
3.4 The Pauli X Gate (The NOT / Flip Gate) . . . 11
3.5 The Hadamard Gate (Creating Superposition) . . . 12
3.6 Phase Gates: Z, S, T, and Rz . . . 13

4 Multi-Qubit Systems and Quantum Circuits **15**
4.1 Tensor Products . . . 15
4.2 Quantum Circuits . . . 15
4.3 Single-Qubit Operations in Multi-Qubit Registers . . . 16
4.4 Measuring a Qubit in a Multi-Qubit Register . . . 16
4.5 Controlled-NOT (CNOT) Gate . . . 16
4.6 Controlled-U Operations . . . 17
4.7 Phase Kickback . . . 17
4.8 Summary: Multi-Qubit Gates . . . 18

5 Entanglement & Interference **19**
5.1 Quantum Entanglement . . . 19
5.1.1 Bell State . . . 19
5.1.2 Mathematical . . . 19
5.2 Quantum Interference . . . 21

6 Quantum Subroutines **24**
6.1 Quantum Fourier Transform . . . 24
6.1.1 Binary Forms and Factorization . . . 25
6.1.2 Integer Truncation . . . 25
6.1.3 Phase Expansion Example . . . 25
6.1.4 The Final Equations . . . 26
6.2 Quantum Phase Estimation . . . 28
6.2.1 Step 1: Create a Superposition . . . 28
6.2.2 Step 2: Apply Controlled Powers of U . . . 29
6.2.3 Step 3: Use the Inverse QFT . . . 29
6.2.4 Step 4: Measure and Recover the Eigenvalue . . . 29
6.3 Grover's Algorithm . . . 31
6.3.1 Step 1: State Preparation . . . 31

6.3.2 Step 2: Oracle . . . 31
6.3.3 Step 3: Diffusion Operator . . . 32
6.3.4 Step 4: Iterate . . . 33
6.4 Quantum Amplitude Estimation . . . 34
6.4.1 The 2D Picture: Good vs. Bad . . . 34
6.4.2 The Grover Operator as a "Rotator" . . . 35
6.4.3 The Clock Analogy (How We Measure . . . 35
6.4.4 Reading the Rotation with Quantum Phase Estimation . . . 35
6.4.5 The Quadratic Speedup (Without the Heavy Math) . . . 36
6.4.6 Bringing It Back to Grover . . . 36

7 Solving Linear Systems **37**
7.1 HHL Algorithm . . . 37
7.2 Initial Setup . . . 37
7.3 Eigendecomposition . . . 38
7.4 Algorithm Workflow . . . 39
7.5 The Complete State Evolution . . . 42
7.6 Key Takeaways . . . 42

8 Shor's Algorithm **43**
8.1 Motivation . . . 43
8.2 The Mathematics of Factoring . . . 43
8.3 Shor's Algorithm: Finding the Period . . . 44
8.3.1 Step 1: State Preparation . . . 44
8.3.2 Step 2: Modular Exponentiation (The Oracle) . . . 45
8.3.3 Step 3: Conceptual Measurement . . . 45
8.3.4 Step 4: The Quantum Fourier Transform (QFT) . . . 45
8.3.5 Step 5: Measurement and Continued Fractions . . . 46
8.4 Extracting the Prime Factors . . . 46
8.5 Current Status . . . 46

9 References **47**

A Mathematical Foundations **48**
A.1 Complex Numbers . . . 48
A.2 Linear Algebra Basics . . . 48
A.3 The Tensor Product ($\otimes$) . . . 48
A.4 Eigenvalues and Eigenvectors . . . 49

1 Motivation

1.1 Bits

In classical computing, a **bit** (binary digit) is the smallest unit of information. It is a two-state system, meaning it can only exist in one of two possible states: 0 or 1. This is analogous to a light switch being either *off* or *on*, or a coin showing either *heads* or *tails*.

If we have n bits, the number of distinct bit-strings they can represent grows **exponentially**. Specifically, n bits can represent 2^n unique combinations. For example:

- 1 bit represents $2^1 = 2$ states: $(0), (1)$.
- 2 bits represent $2^2 = 4$ states: $(00), (01), (10), (11)$.
- 3 bits represent $2^3 = 8$ states: $(000), (001), \ldots, (111)$.

However, there is a critical catch that defines classical computing:

At any given moment, a classical register (group of bits) can hold exactly one of these 2^n possibilities. If you have a 64-bit computer, it can count through 2^{64} numbers, but it only stores a single, definite integer (e.g., $010110 \ldots 101$) at any instant.

Number of Bits (n)	**Total Possible States (2^n)**	**Actual State Held at Time t**
1	2	Either 0 **or** 1
2	4	Exactly one of 00, 01, 10, or 11
n	2^n	Exactly one specific n-bit string

Table 1: Classical bits represent a vast number of possibilities, but only one reality at a time.

1.2 Limitations of Classical Computers

This "one-at-a-time" constraint imposes fundamental limits on how we solve complex problems. Consider a computational search problem. You have N total inputs, but only one specific input x satisfies your condition. A classical machine has no magical shortcut. In the worst case, it must check every single input, one after another, until it finds x.

For many critical problems (like breaking encryption or simulating large molecules), the number of inputs scales as $N = 2^n$, where n is the problem size. This creates exponential growth. If $n = 1000$, then 2^{1000} is a number so large, it dwarfs the total number of atoms in the observable universe. Even with supercomputers running for billions of years, these problems remain practically impossible for classical hardware. We have hit a mathematical ceiling.

Beyond the mathematical wall, there is also a physical wall. For decades, Moore's Law drove computing progress by shrinking transistors. Today, transistors are approaching the scale of individual atoms (around 2 nanometers).

At this size, a phenomenon called **quantum tunneling** occurs: electrons can spontaneously leak across insulating barriers. This leakage generates excessive heat and makes binary states (0 and 1) unreliable. In short, *we cannot keep making classical transistors smaller forever*, and we are rapidly approaching the physical limits of silicon.

1.3 The Quantum Promise

Quantum computing offers a radical departure from the classical "one-state-at-a-time" model. Instead of classical bits, we work with **qubits** (quantum bits). While a classical bit must be definitively 0 or 1, a qubit can exist in a **superposition** of both states simultaneously.

This allows quantum computers to solve certain specific problems exponentially faster than classical computers. While they will never replace your laptop for sending emails or editing spreadsheets, they hold immense promise for specific problems.

2 The Qubit: Mathematical and Geometric Foundations

Like a classical bit, a qubit can take the basic two values: 0 or 1.

To represent them, a qubit uses two basic reference states, also known as the **computational basis states**, which are denoted by unit vectors:

$$|0\rangle = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \qquad |1\rangle = \begin{bmatrix} 0 \\ 1 \end{bmatrix}.$$

The vertical bar and angle bracket is called a *ket*, which comes from Dirac or bra-ket notation from quantum physics. This is just used to denote a quantum state.

2.1 Superposition

A classical bit is either 0 or 1. However, a qubit can be in a **superposition**, where it is **neither** definitively 0 nor 1.

Superposition

Here's an analogy: think about a standard coin. When you flip it and read it, it can either be heads or tails. But what is it at the moment it is spinning in the air? Before it lands and you "measure" it, the qubit exists in a fluid state where it is a combination of both heads and tails simultaneously.

A general qubit state $\psi\rangle$ can be written as a linear combination of these two basis states:

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle.$$

Equivalently, this can be written as the complex column vector

$$|\psi\rangle = \begin{bmatrix} \alpha \\ \beta \end{bmatrix} \in \mathbb{C}^2.$$

Before measurement, the qubit exists in a blend of both basis states, characterized entirely by the ratio of α to β.

2.2 Probabilities and Amplitudes

The numbers α and β are called **probability amplitudes**. These are **complex** numbers that can be negative or imaginary. They represent the **probability** that the measurement of a quantum system will yield a specific result. The amplitude α belongs to the basis state $|0\rangle$, and β belongs to the basis state $|1\rangle$.

Why Complex Numbers?

This involves a lot of quantum physics (which we won't dive deep into), but what you need to know is that quantum states can be represented by wave functions. The governing Schrödinger equation is a wave equation, and these waves are fundamentally described by complex exponentials ($e^{i\phi}$).

Classical probabilities are just positive numbers that add together. Quantum amplitudes, however, must be able to **cancel each other out** for reasons we will discuss later. This cancellation requires both a magnitude and a sign/direction, which complex numbers naturally provide.

To get the actual classical probability of measuring the qubit in state $|0\rangle$ or $|1\rangle$, we use **Born's rule**. This is taking the squared magnitude of the amplitude:

$$P(0) = |\alpha|^2, \quad P(1) = |\beta|^2$$

Furthermore, to represent a valid qubit, the amplitudes must satisfy the **normalization** condition that makes the total probability equal 1.

$$|\alpha|^2 + |\beta|^2 = 1.$$

2.3 Measuring a Qubit

Once a measurement is made, the original superposition is destroyed. The qubit **collapses** into either $|0\rangle$ or $|1\rangle$. This act fundamentally alters the qubit, and any subsequent measurement will return the same value unless additional quantum operations are performed.

This highlights an important distinction between classical and quantum computation. A classical bit can be read directly, revealing its stored value with complete certainty. A quantum state, however, cannot be read directly. Instead, each measurement provides only a single random sample drawn relative to the state's probability distribution.

A single measurement might produce either $|0\rangle$ or $|1\rangle$, but it gives no information about the probability amplitudes themselves. To estimate the underlying probabilities, the same quantum circuit must be prepared and measured repeatedly.

As an example, suppose a qubit is prepared in the state

$$|\psi\rangle = \frac{\sqrt{3}}{2}|0\rangle + \frac{1}{2}|1\rangle.$$

then

$$P(0) = \left|\frac{\sqrt{3}}{2}\right|^2 = \frac{3}{4}, \qquad P(1) = \left|\frac{1}{2}\right|^2 = \frac{1}{4}.$$

Over many independent runs (called *shots*), approximately 75% of the measurements will produce 0, while approximately 25% will produce 1.

As the number of shots increases, these observed frequencies converge toward the true probabilities.

This process of repeating a quantum computation many times to estimate measurement probabilities is known as **statistical sampling**. Because quantum measurements are inherently probabilistic, many quantum algorithms rely on repeated sampling to extract useful information from the final quantum state.

2.4 The Scaling Table: Classical vs. Quantum

A common trap is thinking n qubits are just "n bits but faster." The reality is more subtle.

Property	n **Classical Bits**	n **Qubits**
Mathematical State	A single definite string (e.g., 0101).	A superposition of all 2^n strings.
Information held	Exactly 1 specific combination.	2^n complex amplitudes simultaneously.
On Measurement	Reads the stored string.	Collapses to just 1 random string.

Table 2: The critical distinction: Qubits explore exponentially many paths, but we only get to see one classical outcome.

This is why quantum computing is powerful *and* tricky. The superposition gives us massive parallel "exploration," but the measurement destroys the rest of the information.

2.5 Deriving the Bloch Sphere (From Cartesian to Spherical)

We now build the famous geometric representation.

Step 1: Cartesian to Polar. Instead of using standard Cartesian coordinates (x, y) where x represents real numbers and y represents imaginary numbers, quantum states are expressed in polar form using Euler's formula:

$$re^{i\theta}$$

Using polar form, an arbitrary qubit can be written as

$$|\psi\rangle = r_0 e^{i\phi_0}|0\rangle + r_1 e^{i\phi_1}|1\rangle$$

This state appears to require four real parameters: the two magnitudes (r_0, r_1) and the two *phase* angles (ϕ_0, ϕ_1). However, four dimensions are hard to visualize intuitively (we would need a 4D space!), so we use physics and math to eliminate two of them.

Step 2: Factor out the global phase. Substitute these into the state vector:

$$|\psi\rangle = e^{i\phi_0}\left(r_0|0\rangle + r_1 e^{i(\phi_1-\phi_0)}|1\rangle\right)$$

The factor $e^{i\phi_0}$ in front is called the **global phase**. Crucially, it has no physical effect. If you multiply an entire quantum state by $e^{i\phi}$, all probabilities ($|\alpha|^2$ and $|\beta|^2$) remain unchanged. We are free to drop it.

This leaves us with a state determined by two real parameters: the magnitudes r_0, r_1 and the relative phase difference $\phi = \phi_1 - \phi_0$.

Step 3: Applying normalization. Recall that $|\alpha|^2 + |\beta|^2 = r_0^2 + r_1^2 = 1$. This is the equation of a unit circle in 2D. We can parametrize this using trigonometry:

$$r_0 = \cos\left(\frac{\theta}{2}\right), \quad r_1 = \sin\left(\frac{\theta}{2}\right)$$

Step 4: The final standard form. Substituting this back in, we get the standard parameterization of a qubit:

$$|\psi\rangle = \cos\left(\frac{\theta}{2}\right)|0\rangle + e^{i\phi}\sin\left(\frac{\theta}{2}\right)|1\rangle$$

By applying these constraints, every pure qubit state can instead be described using only two real parameters: the angles θ and ϕ. These two angles become the spherical coordinates used to represent qubits on the Bloch sphere.

2.6 The Bloch Sphere

The state of any single qubit can be mapped to the surface of a 3D unit sphere. This is called the **Bloch Sphere**, and it is the single most useful tool for visualizing a qubit.

The Earth Analogy

To understand it, imagine our planet Earth.

- The **North Pole** represents the state $|0\rangle$.
- The **South Pole** represents the state $|1\rangle$.

Just like Earth has latitude and longitude, the Bloch sphere uses two angles:

- **Polar Angle** (θ): This is your "latitude." It controls the magnitude, or the probability of measuring 0 versus 1.
- **Azimuthal Angle** (ϕ): This is your "longitude." It controls the complex phase difference between the two states. Measured counterclockwise from the X-axis in the XY-plane.

With $0 \leq \theta \leq \pi$ and $0 \leq \phi \leq 2\pi$, **every** possible qubit state can be geometrically represented by exactly one unique point on this sphere.

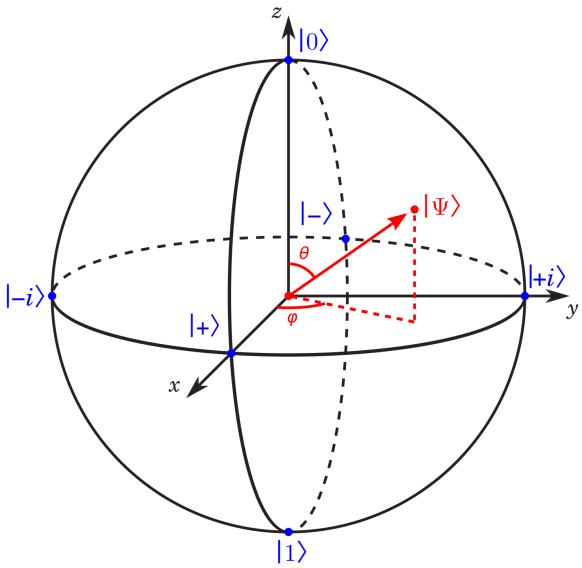


Figure 1: The Bloch sphere visualized as a globe.

Why the Equator is 50/50 Superposition

Recall the standard form of a qubit state from our previous derivation:

$$|\psi\rangle = \cos\left(\frac{\theta}{2}\right)|0\rangle + e^{i\phi}\sin\left(\frac{\theta}{2}\right)|1\rangle$$

Now, think about Earth's Equator. Any point on the equator has a latitude of exactly $0°$, which in radians is $\theta = \pi/2$.

Let us plug that into our equation:

$$|\psi\rangle = \cos\left(\frac{\pi}{4}\right)|0\rangle + e^{i\phi}\sin\left(\frac{\pi}{4}\right)|1\rangle$$

Since $\cos(\pi/4) = \sin(\pi/4) = \frac{1}{\sqrt{2}}$, we get:

$$|\psi\rangle = \frac{1}{\sqrt{2}}|0\rangle + \frac{e^{i\phi}}{\sqrt{2}}|1\rangle$$

When we calculate the measurement probabilities:

$$P(0) = \left|\frac{1}{\sqrt{2}}\right|^2 = \frac{1}{2}, \quad P(1) = \left|\frac{e^{i\phi}}{\sqrt{2}}\right|^2 = \frac{1}{2}$$

Why $\theta/2$?

We want the physical latitude (θ) on the Bloch sphere to range from 0 (North Pole = $|0\rangle$) to π (South Pole = $|1\rangle$). If we used just θ without the half, the equator ($\theta = \pi/2$) would give probabilities $P(0) = \cos^2(\pi/2) = 0$, which is wrong.

By using $\theta/2$, the equator becomes $\theta = \pi/2$, giving $P(0) = \cos^2(\pi/4) = 1/2$ and $P(1) = \sin^2(\pi/4) = 1/2$. The half-angle ensures that **perfect 50/50 superpositions live exactly on the equator** of our 3D globe.

This is the key insight: Every single point on the Bloch sphere's equator represents a perfect 50/50 superposition of $|0\rangle$ and $|1\rangle$. The only thing that changes as you walk along the equator (changing ϕ) is the *relative phase* between the two states, not the measurement odds.

Bases: Different Axes, Different Measurements

The poles do not always have to be $|0\rangle$ and $|1\rangle$.

A **basis** is simply a set of two opposite points on the sphere that represent distinct, perfectly distinguishable measurement outcomes. We name each basis after the axis it lies on:

- **Z-basis:** $\{|0\rangle, |1\rangle\}$. These lie on the North and South poles (the Z-axis). This is the default measurement we use in most quantum circuits.
- **X-basis:** $\{|+\rangle, |-\rangle\}$. These lie on the equator, specifically on the positive and negative X-axis. Here, $|+\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$ and $|-\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)$.
- **Y-basis:** $\{|i\rangle, |-i\rangle\}$. These lie on the equator, specifically on the positive and negative Y-axis. Here, $|i\rangle = \frac{1}{\sqrt{2}}(|0\rangle + i|1\rangle)$ and $|-i\rangle = \frac{1}{\sqrt{2}}(|0\rangle - i|1\rangle)$.

2.7 Circle Notation

In a system of n qubits, there are 2^n possible classical states. Using bloch spheres to representing these systems can become overwhelming and complicated. This is why we use circle notation. Circle notation represents each possible quantum state as an individual circle, making it easier to visualize.

- The shaded region inside the circle represents the probability of the qubit collapsing into that state upon measurement. A completely filled circle means a 100% probability; an empty circle means 0%.
- A line inside the circle points to a specific angle, representing the relative phase (ϕ) of that state's complex amplitude. This can take any value from 0° to 360°.
  - Pointing straight up (12 o'clock) means a phase of 0 (a positive real number).
  - Pointing to the right (3 o'clock) represents $+i$.
  - Pointing straight down (6 o'clock) represents a negative real number (a phase of π).
  - Pointing to the left (9 o'clock) represents $-i$.

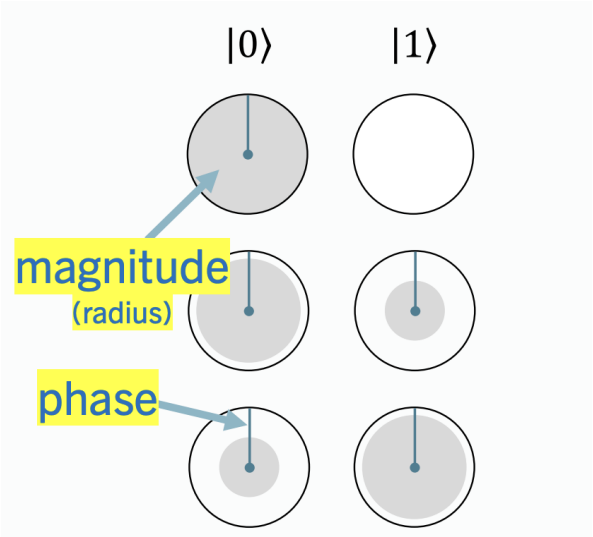


Figure 2: Circle Notation

3 Single-Qubit Quantum Gates

3.1 What is a Quantum Gate?

A quantum gate is an operation that transforms a qubit state into another qubit state. If we have an input state $|\psi\rangle$, the gate applies a matrix U to produce an output state $|\psi'\rangle$:

$$U|\psi\rangle = |\psi'\rangle$$

However, we cannot just use any matrix. Quantum mechanics imposes strict rules on what U is allowed to be. This brings us to the most important property of quantum gates: **unitarity**.

3.2 Why Unitary?

Rule 1: Probability Must Be Conserved

Recall the normalization condition from Section 2:

$$|\alpha|^2 + |\beta|^2 = 1$$

This means the total probability of finding the qubit in either state must always equal exactly 1. A gate must never change this total probability.

If we apply a gate U to a qubit, the new amplitudes are transformed. The new state $|\psi'\rangle$ must also satisfy:

$$|\alpha'|^2 + |\beta'|^2 = 1$$

In linear algebra, the norm (length) of the state vector is $\sqrt{\langle\psi|\psi\rangle}$. This norm must stay exactly the same after applying the gate.

$$\langle\psi'|\psi'\rangle = \langle\psi|U^\dagger U|\psi\rangle = \langle\psi|\psi\rangle = 1$$

For this equation to hold true for *every* possible input state $|\psi\rangle$, the gate matrix U must satisfy:

$$U^\dagger U = I$$

Any matrix that satisfies this condition is called a **unitary matrix**.

Unitary Matrix

A complex square matrix whose inverse is equal to its conjugate transpose ($U^{-1} = U^\dagger$). The easiest way to verify this is to multiply the matrix by its conjugate transpose; if the result is the Identity matrix (I), it is unitary.

$$\underbrace{\begin{bmatrix} 1 & 0 \\ 0 & i \end{bmatrix}}_{U} \times \underbrace{\begin{bmatrix} 1 & 0 \\ 0 & -i \end{bmatrix}}_{U^\dagger \text{ (Conjugate Transpose)}} = \underbrace{\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}}_{I \text{ (Identity Matrix)}}$$

Because $U \times U^\dagger = I$, the matrix U is unitary.

The dagger (†) means we do two things to the matrix:

1. Take the **complex conjugate** of every entry (swap i for $-i$).
2. **Transpose** the matrix (flip it over the diagonal).

This is simply the mathematical way to construct the inverse of a quantum gate.

Rule 2: Quantum Gates Must Be Reversible

Unlike classical logic gates (like AND or OR) which often lose information, quantum gates must be perfectly reversible.

If you apply a gate U, you must always be able to apply another operation that **undoes** it and returns the qubit to its original state.

For example, applying the gate U and then its inverse U^{-1} brings you back to where you started:

$$U^{-1}U = I$$

For unitary matrices, the inverse is always $U^{-1} = U^\dagger$. This is why the condition $U^\dagger U = I$ appears again. Unitarity guarantees that no information is ever lost.

3.3 Visualizing Unitary Operations

Think of a unitary matrix as a **rigid rotation or reflection** of the Bloch sphere.

Just like rotating a globe does not change the physical size of the Earth, a unitary operation rotates the qubit's state vector but never stretches or shrinks it. The vector always remains exactly on the surface of the unit sphere.

Property	Classical Gates (AND, OR)	Quantum Gates (Unitary)
Information Loss	Often loses bits (irreversible).	Never loses information (reversible).
Inverse	Does not always exist.	Always exists: $U^{-1} = U^\dagger$.

Table 3: The fundamental difference between classical logic and quantum unitarity.

3.4 The Pauli X Gate (The NOT / Flip Gate)

The simplest quantum gate is the **Pauli-X** gate, often called the **NOT** or **flip** gate. Its matrix form is:

$$X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$$

To see what it does, apply it to the basis states:

$$X|0\rangle = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \end{pmatrix} = \begin{pmatrix} 0 \\ 1 \end{pmatrix} = |1\rangle$$

$$X|1\rangle = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} 0 \\ 1 \end{pmatrix} = \begin{pmatrix} 1 \\ 0 \end{pmatrix} = |0\rangle$$

It flips $|0\rangle$ to $|1\rangle$ and $|1\rangle$ back to $|0\rangle$. This is the direct quantum analogue of the classical NOT gate.

$$X \begin{bmatrix} \alpha \\ \beta \end{bmatrix} = \begin{bmatrix} \beta \\ \alpha \end{bmatrix} = \beta|0\rangle + \alpha|1\rangle$$

On the Bloch sphere, the X gate performs a **180° rotation around the X-axis**.

- The North Pole ($|0\rangle$) rotates to the South Pole ($|1\rangle$).
- The South Pole ($|1\rangle$) rotates to the North Pole ($|0\rangle$).
- Points on the equator (like $|+\rangle$) stay on the equator but rotate to their opposite point on the X-axis.

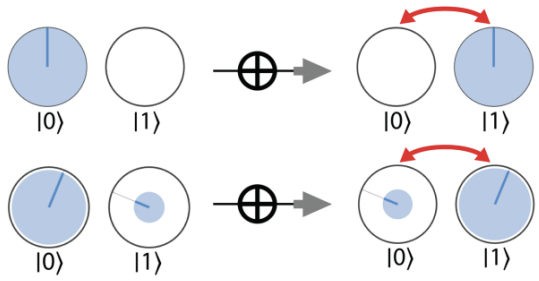


Figure 3: Flip Gate Circle Notation

Why "Pauli"?

The X gate is part of a set of three matrices (X, Y, Z) called the **Pauli matrices**. They form the foundation of all single-qubit rotations. In the next section, we will meet Z (phase flip) and later Y. For now, think of X as the "swap" operation.

3.5 The Hadamard Gate (Creating Superposition)

The **Hadamard gate** (often just called the H gate) is arguably the most important single-qubit gate. It is our primary tool for creating superposition. Its matrix is:

$$H = \frac{1}{\sqrt{2}}\begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$$

Applying it to the basis states:

$$H|0\rangle = \frac{1}{\sqrt{2}}\begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}\begin{pmatrix} 1 \\ 0 \end{pmatrix} = \frac{1}{\sqrt{2}}\begin{pmatrix} 1 \\ 1 \end{pmatrix} = \frac{|0\rangle + |1\rangle}{\sqrt{2}} = |+\rangle$$

$$H|1\rangle = \frac{1}{\sqrt{2}}\begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}\begin{pmatrix} 0 \\ 1 \end{pmatrix} = \frac{1}{\sqrt{2}}\begin{pmatrix} 1 \\ -1 \end{pmatrix} = \frac{|0\rangle - |1\rangle}{\sqrt{2}} = |-\rangle$$

For a general state $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$:

$$H|\psi\rangle = \frac{\alpha+\beta}{\sqrt{2}}|0\rangle + \frac{\alpha-\beta}{\sqrt{2}}|1\rangle$$

The Hadamard takes a definite basis state and spreads it into a **perfect 50/50 superposition**. On the Bloch sphere, it performs a 180° rotation around the diagonal axis $(1, 0, 1)$, effectively swapping the Z-axis with the X-axis. The North Pole ($|0\rangle$) maps to $|+\rangle$ on the equator; the South Pole ($|1\rangle$) maps to $|-\rangle$.

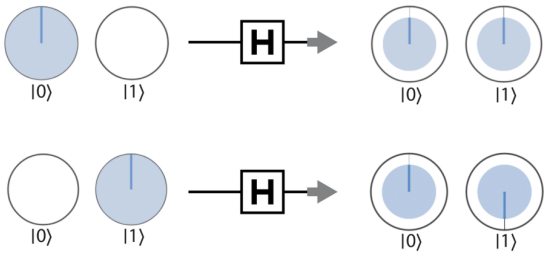


Figure 4: The H gate splits one filled circle into two half-filled circles.

The Hadamard is its own inverse

One property of the H gate is that applying it twice brings you back to where you started:

$$H^2 = I$$

3.6 Phase Gates: Z, S, T, and Rz

The X gate flips probabilities. The Hadamard spreads them. Phase gates do neither—they change only the *complex phase* without altering measurement probabilities. In circle notation, the filled area stays exactly the same; only the phase angle rotates.

The General Phase Gate: Rz(θ)

Arbitrary Z-rotation:

$$Rz(\theta) = \begin{pmatrix} 1 & 0 \\ 0 & e^{i\theta} \end{pmatrix}, \quad Rz(\theta)|0\rangle = |0\rangle, \quad Rz(\theta)|1\rangle = e^{i\theta}|1\rangle$$

For a general state: $Rz(\theta)(\alpha|0\rangle + \beta|1\rangle) = \alpha|0\rangle + e^{i\theta}\beta|1\rangle$.

On the Bloch sphere, all phase gates perform rotations around the Z-axis. The poles stay fixed; points on the equator shift their longitude. In circle notation, the right circle's phase rotates by the specified angle while its filled area remains unchanged.

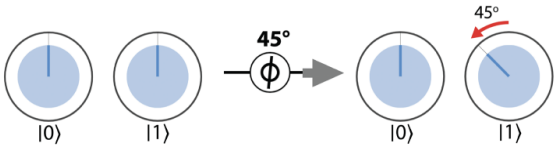


Figure 5: Phase gates change only the phase of the $|1\rangle$ circle.

Why Phase Matters

On their own, phase gates seem useless—they do not change measurement probabilities. However, when combined with Hadamard gates, phases become the engine of quantum algorithms. They control whether amplitudes *constructively interfere* (add up) or *destructively interfere* (cancel out). This interference is what gives quantum computers their power.

Gate	**Matrix**	**Phase on $\|1\rangle$**	**Relationship**
Z	$\begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$	π (180°)	$Z = S^2$
S	$\begin{pmatrix} 1 & 0 \\ 0 & i \end{pmatrix}$	$\pi/2$ (90°)	$S = T^2$
T	$\begin{pmatrix} 1 & 0 \\ 0 & e^{i\pi/4} \end{pmatrix}$	$\pi/4$ (45°)	$T = Rz(\pi/4)$
$Rz(\theta)$	$\begin{pmatrix} 1 & 0 \\ 0 & e^{i\theta} \end{pmatrix}$	θ (arbitrary)	General case

Table 4: The family of phase gates. Each rotates the phase of $|1\rangle$ by a specific angle.

4 Multi-Qubit Systems and Quantum Circuits

As useful as single qubits can be, they're much more powerful (and intriguing) in groups. When our quantum computer has access to more than one qubit, we can make use of a second powerful quantum phenomenon known as *entanglement*.

4.1 Tensor Products

While a single qubit exists in a 2D complex vector space, combining n qubits requires combining their state spaces. We use the tensor product ($\otimes$) to mathematically represent these joint multi-qubit states.

When qubits are independent, we write their combined state as a product. For example, if you have 5 qubits all initialized to the $|1\rangle$ state, you write it out fully as:

$$|1\rangle \otimes |1\rangle \otimes |1\rangle \otimes |1\rangle \otimes |1\rangle = |11111\rangle$$

or equivalently:

$$|1\rangle^{\otimes 5} = |11111\rangle$$

Tensor Product of Qubits

The tensor product ($\otimes$) multiplies every element of the first vector by the entire second vector. This is how we mathematically combine individual qubits to represent a multi-qubit system.

For example, let's look at the tensor product of state $|1\rangle$ and state $|0\rangle$:

$$|1\rangle = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \quad |0\rangle = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

$$|1\rangle \otimes |0\rangle = \begin{bmatrix} 0 \\ 1 \end{bmatrix} \otimes \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \times \begin{bmatrix} 1 \\ 0 \end{bmatrix} \\ 1 \times \begin{bmatrix} 1 \\ 0 \end{bmatrix} \end{bmatrix} = \begin{bmatrix} 0 \times 1 \\ 0 \times 0 \\ 1 \times 1 \\ 1 \times 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \end{bmatrix} = |10\rangle$$

The resulting 4-dimensional column vector maps perfectly to the computational basis state $|10\rangle$.

4.2 Quantum Circuits

Just like we can draw a classical circuit diagram consisting of bits and logic gates, we can draw quantum circuit diagrams consisting of qubits and quantum gates.

Each quantum circuit consists of:

- **Horizontal wires:** Each wire represents one qubit. The top wire is qubit 0, the next is qubit 1, and so on.
- **Time flow:** Time moves from left to right. The leftmost operations happen first.
- **Initialization:** The left side shows the initial state of each qubit (usually $|0\rangle$).
- **Gates:** Each gate is a box placed on one or more wires, updating the state of the qubits it touches.

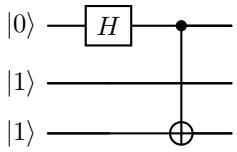


Figure 6: A simple quantum circuit with three qubits. Time flows left to right.

4.3 Single-Qubit Operations in Multi-Qubit Registers

What happens (in circle notation) when we apply single-qubit operations such as NOT, HAD, and PHASE to a multi-qubit register?

The only difference from the single-qubit case is that the circles are operated on in certain **operator pairs** specific to the qubit that the operation acts on.

To identify a qubit's operator pairs, match each circle with the one whose value differs by the qubit's bit-value. For example, if we are operating on qubit 0x4, then each pair will include circles whose values differ by exactly 4.

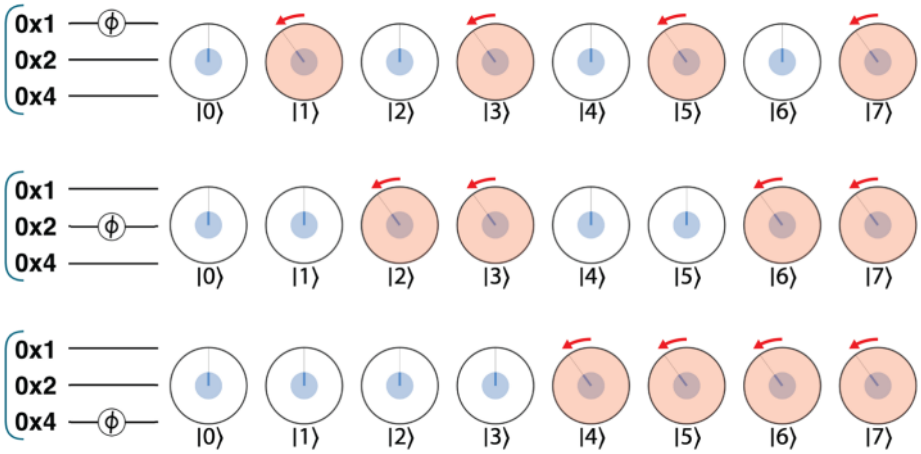


Figure 7: The NOT operation swaps values in each of the qubit's operator pairs.

4.4 Measuring a Qubit in a Multi-Qubit Register

Following a measurement, the state of our multi-qubit register will change to reflect which outcome occured. All circles that do not agree with the result will be eliminated, as shown below.

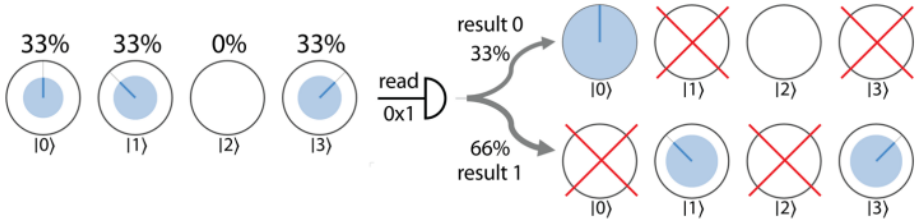


Figure 8: Reading one qubit in a multi-qubit register.

4.5 Controlled-NOT (CNOT) Gate

The CNOT (Controlled-NOT) gate is the fundamental two-qubit gate. It is the quantum analogue of the classical XOR gate. It can also be thought of as the quantum equivalent of an "if-statement" in

programming.

The **control** qubit is a black dot, while the **target** qubit is the gate itself.

- If the control qubit is $|0\rangle$, nothing happens.
- If the control qubit is $|1\rangle$, it flips the target qubit.
- If the control qubit is in a superposition when it hits the CNOT, the gate forces the target qubit into a superposition that is perfectly tied to the control. This creates an entangled Bell State.

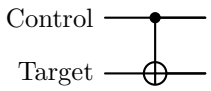


Figure 9: The CNOT gate: a solid dot on the control line and a crossed circle on the target line.

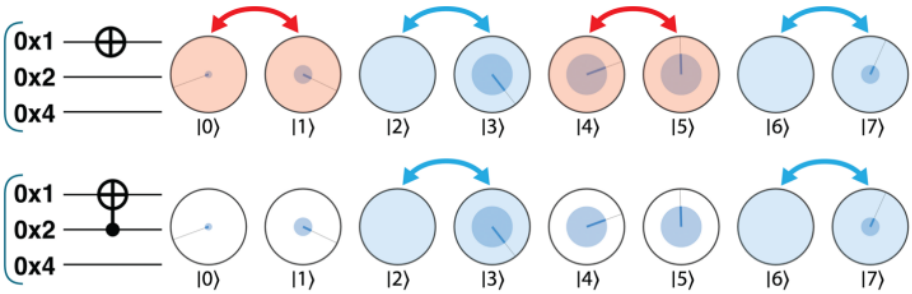


Figure 10: NOT versus CNOT in operation.

We can see that the essential operation of CNOT is the same as that of NOT, only more selective—applying the NOT operation only to values whose binary representations (in this example) have a 1 in the second bit (010=2, 011=3, 110=6, and 111=7).

Like the NOT operation, CNOT is its own inverse—applying the CNOT operation twice will return a multi-qubit register to its initial state.

4.6 Controlled-U Operations

The CNOT is actually a special case of a more general operation. We can apply *any* single-qubit gate U to the target qubit, controlled by the control qubit.

This is called a **Controlled-U** gate:

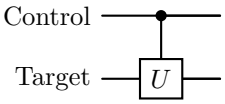


Figure 11: A Controlled-U gate. The target qubit has U applied only if the control is $|1\rangle$.

Important note: The control qubit *does not change* when we apply a Controlled-U gate. Only the target qubit is affected.

4.7 Phase Kickback

Phase kickback is a useful effect that occurs when a controlled gate applies a phase to its target. Although the gate acts on the target qubit, the phase can appear on the **control qubit** instead.

The simplest example uses a Controlled-Z gate. Suppose the control begins in the state $|+\rangle$ and the target begins in $|1\rangle$:

$$|+\rangle|1\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}}|1\rangle.$$

The Controlled-Z does nothing to the $|0\rangle$ part of the control. For the $|1\rangle$ part, it applies Z to the target. Because $Z|1\rangle = -|1\rangle$, we obtain

$$\frac{|0\rangle|1\rangle + |1\rangle Z|1\rangle}{\sqrt{2}} = \frac{|0\rangle|1\rangle - |1\rangle|1\rangle}{\sqrt{2}} = |-\rangle|1\rangle.$$

Notice what happened: the target is still $|1\rangle$, but the control changed from $|+\rangle$ to $|-\rangle$. The minus sign introduced by Z has "kicked back" onto the control.

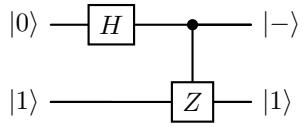


Figure 12: Phase kickback using a Controlled-Z gate. The target remains $|1\rangle$, while the control acquires a minus sign.

More generally, suppose U adds a phase $e^{i\phi}$ to a target state $|u\rangle$, so that $U|u\rangle = e^{i\phi}|u\rangle$. If the control is in a superposition, then

$$\frac{|0\rangle + |1\rangle}{\sqrt{2}}|u\rangle \quad \longrightarrow \quad \frac{|0\rangle + e^{i\phi}|1\rangle}{\sqrt{2}}|u\rangle.$$

The target state is unchanged, while the phase $e^{i\phi}$ is now stored in the relative phase of the control qubit. This is the basic idea behind phase kickback.

Key Idea

Phase kickback does not flip the control qubit from $|0\rangle$ to $|1\rangle$ or vice versa. Instead, it changes the *relative phase* between the control's $|0\rangle$ and $|1\rangle$ components. This effect is used in important algorithms such as quantum phase estimation.

4.8 Summary: Multi-Qubit Gates

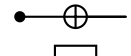
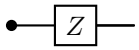
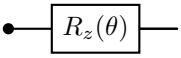
Gate	Effect	Circuit Symbol
CNOT	Flips target if control is $\|1\rangle$	
Controlled-Z	Applies Z to target if control is $\|1\rangle$	
Controlled-Rz(θ)	Applies $Rz(\theta)$ to target if control is $\|1\rangle$	

Table 5: Common multi-qubit gates and their circuit symbols.

5 Entanglement & Interference

We will now study two strange quantum phenomemon that are crucial to our study of quantum computing.

5.1 Quantum Entanglement

Quantum entanglement is a fundamental phenomenon in quantum physics where two or more particles become linked in such a way that the state of one particle instantly determines the state of the other, no matter how far apart they are. This mysterious connection puzzled even Einstein, who famously called it *"spooky action at a distance."*

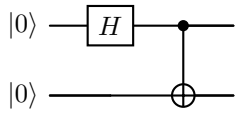
An entangled state in quantum computing is a special type of multi-qubit state in which the individual qubits do not have independent values. Instead, their states are linked: measuring one qubit instantly reveals information about the others.

5.1.1 Bell State

The most fundamental multi-qubit circuit is the creation of a Bell Pair (an entangled state).

The Algorithm for a Bell Pair:

- Hadamard on Qubit A: Put the first qubit into a perfect 50/50 superposition ($|0\rangle$ and $|1\rangle$).
- CNOT (A controls B): Apply a CNOT gate where Qubit A is the control and Qubit B (initialized to 0) is the target.



Qubit B now only flips if Qubit A is evaluated as a 1. Because Qubit A is in superposition, both qubits are now locked together in a unified state:

$$|\psi\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$$

If you measure Qubit A and get a 1, Qubit B is instantly mathematically guaranteed to also be a 1, no matter where it is physically located.

5.1.2 Mathematical

A state is entangled if it **cannot** be factored into the tensor product of individual qubit vectors. In linear algebra terms, this means the full state vector cannot be separated into two smaller column vectors.

When we combine two independent qubits, we use the tensor product ($\otimes$), creating a 4D vector space ($\mathbb{C}^4$):

$$|00\rangle = |0\rangle \otimes |0\rangle = \begin{pmatrix} 1 \\ 0 \\ 0 \\ 0 \end{pmatrix}, \quad |11\rangle = |1\rangle \otimes |1\rangle = \begin{pmatrix} 0 \\ 0 \\ 0 \\ 1 \end{pmatrix}$$

The $|\Phi^+\rangle$ Bell state (maximally entangled state) is defined as:

$$|\Phi^+\rangle = \frac{1}{\sqrt{2}}|00\rangle + \frac{1}{\sqrt{2}}|11\rangle = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 0 \\ 0 \\ 1 \end{pmatrix}$$

If these two qubits were not entangled, we could separate them. This means we could write $|\Phi^+\rangle$ as the tensor product of two independent single-qubit states, $|\psi_A\rangle$ and $|\psi_B\rangle$:

$$|\psi_A\rangle = \begin{pmatrix} a \\ b \end{pmatrix} = a|0\rangle + b|1\rangle$$

$$|\psi_B\rangle = \begin{pmatrix} c \\ d \end{pmatrix} = c|0\rangle + d|1\rangle$$

Let's compute their tensor product:

$$|\psi_A\rangle \otimes |\psi_B\rangle = \begin{pmatrix} a \\ b \end{pmatrix} \otimes \begin{pmatrix} c \\ d \end{pmatrix} = \begin{pmatrix} ac \\ ad \\ bc \\ bd \end{pmatrix} = ac|00\rangle + ad|01\rangle + bc|10\rangle + bd|11\rangle$$

Now, we try to set this equal to our Bell State:

$$\begin{pmatrix} ac \\ ad \\ bc \\ bd \end{pmatrix} = \begin{pmatrix} \frac{1}{\sqrt{2}} \\ 0 \\ 0 \\ \frac{1}{\sqrt{2}} \end{pmatrix}$$

This gives us a system of four algebraic equations:

1. $ac = \frac{1}{\sqrt{2}}$
2. $ac = \frac{1}{\sqrt{2}}$
3. $bc = 0$
4. $bd = \frac{1}{\sqrt{2}}$

From equation (2), $ad = 0$, meaning either $a = 0$ or $d = 0$.

- If $a = 0$, then equation (1) becomes $(0)c = \frac{1}{\sqrt{2}}$, which is impossible.
- If $d = 0$, then equation (4) becomes b$(0) = \frac{1}{\sqrt{2}}$, which is also impossible.

Conclusion: Because no such scalars a, b, c, d exist, the state vector cannot be factored into individual qubit states. This mathematical inseparability is exactly what quantum entanglement is.

5.2 Quantum Interference

Before we dive into algorithms, we need to understand the fundamental mechanism that makes them work: **interference**.

Interference is the secret sauce of quantum computing. Without it, quantum computers would be no better than classical probabilistic computers.

Conceptual Understanding: Waves and Probabilities

Imagine dropping two stones into a pond. The ripples spread out and overlap.

- Where the peaks meet peaks, the wave gets **taller** (constructive interference).
- Where a peak meets a trough, the wave **cancels out** (destructive interference).

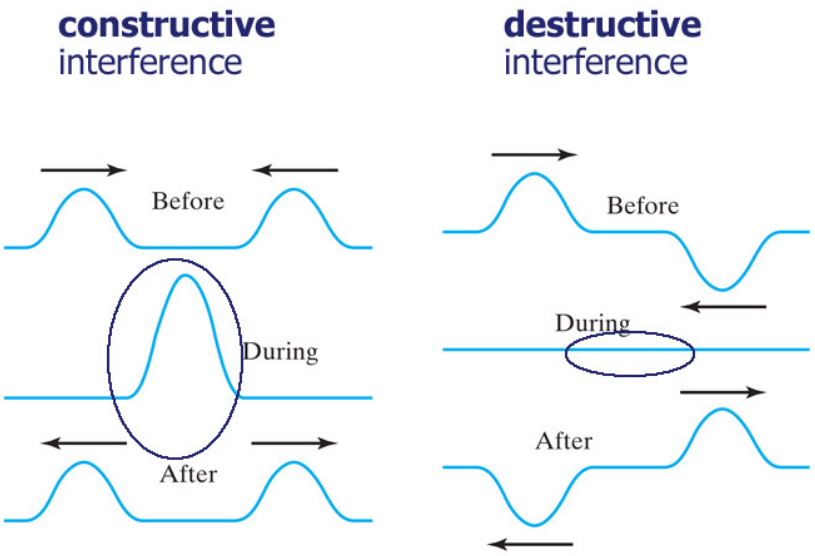


Figure 13: Two sources of waves creating interference patterns.

In quantum computing, we do something similar, but with **probability amplitudes** instead of waves.

Classical probability works like this: if there are two possible paths to an outcome, you add their probabilities. If path A has 50% chance and path B has 50% chance, the total is 100%.

Quantum mechanics works differently: **you add the amplitudes first, then take the square.**

$$P_{\text{total}} = |\alpha_A + \alpha_B|^2$$

This seemingly small difference has enormous consequences.

A Simple Example

Imagine a qubit in a superposition:

$$|\psi\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$$

Both $|0\rangle$ and $|1\rangle$ have amplitude $1/\sqrt{2}$. The probability of measuring $|0\rangle$ is $|1/\sqrt{2}|^2 = 1/2$, and similarly for $|1\rangle$.

Now, suppose we apply a phase gate that adds a relative phase ϕ to the $|1\rangle$ state:

$$|\psi\rangle = \frac{1}{\sqrt{2}}(|0\rangle + e^{i\phi}|1\rangle)$$

The probabilities are still 1/2 and 1/2. The phase changed nothing.

But now apply a Hadamard gate:

$$H|\psi\rangle = \frac{1}{\sqrt{2}}\left(H|0\rangle + e^{i\phi}H|1\rangle\right) = \frac{1}{2}\left[(|0\rangle + |1\rangle) + e^{i\phi}(|0\rangle - |1\rangle)\right]$$

$$= \frac{1}{2}\left[(1 + e^{i\phi})|0\rangle + (1 - e^{i\phi})|1\rangle\right]$$

Now the amplitudes of $|0\rangle$ and $|1\rangle$ *depend on* ϕ:

$$\alpha_0 = \frac{1 + e^{i\phi}}{2}, \quad \alpha_1 = \frac{1 - e^{i\phi}}{2}$$

The probabilities are:

$$P(0) = |\alpha_0|^2 = \left|\frac{1 + e^{i\phi}}{2}\right|^2 = \cos^2\left(\frac{\phi}{2}\right)$$

$$P(1) = |\alpha_1|^2 = \left|\frac{1 - e^{i\phi}}{2}\right|^2 = \sin^2\left(\frac{\phi}{2}\right)$$

The Interference Pattern

Notice what happened:

- When $\phi = 0$: $P(0) = 1$, $P(1) = 0$ (constructive interference for $|0\rangle$).
- When $\phi = \pi$: $P(0) = 0$, $P(1) = 1$ (constructive interference for $|1\rangle$).
- When $\phi = \pi/2$: $P(0) = P(1) = 1/2$ (no interference; both paths equally likely).

The phases that seemed "invisible" before now **determine the measurement outcome**. This is interference at work.

The Mathematical Details

In quantum computing, interference arises whenever a qubit has multiple paths to the same final state.

Consider a general superposition:

$$|\psi\rangle = \sum_i \alpha_i |i\rangle$$

Each amplitude α_i has a magnitude and a phase: $\alpha_i = |\alpha_i|e^{i\phi_i}$.

When we apply operations, the amplitudes from different paths combine. If two paths have amplitudes with the same phase, they constructively interfere:

$$|\alpha + \beta|^2 > |\alpha|^2 + |\beta|^2$$

If they have opposite phases, they destructively interfere:

$$|\alpha - \beta|^2 < |\alpha|^2 + |\beta|^2$$

The Quantum Circuit Perspective

Let us revisit the previous example as a circuit:

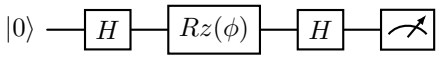


Figure 14: The interference circuit: H, phase shift, H. The phase becomes visible in the measurement.

This circuit demonstrates three stages of interference:

1. **Hadamard (first):** Creates superposition with two paths.
2. **Rz(ϕ):** Adds a phase to one path.
3. **Hadamard (second):** Recombines the paths. The phases determine whether they constructively or destructively interfere.

The Sneaky Trick of Interference

Quantum algorithms exploit interference in a clever way:

1. Create a superposition of all possible inputs.
2. Apply a function that computes all outputs in parallel.
3. **Use interference to make the wrong answers cancel out** and the correct answer **build up**.

This is why quantum computers can solve certain problems exponentially faster than classical computers.

Interference vs. Entanglement

It is important to distinguish interference from entanglement:

- **Interference:** A single-qubit phenomenon. It happens when a qubit's own amplitudes combine.
- **Entanglement:** A multi-qubit phenomenon. It happens when qubits become correlated.

However, they work together. Many quantum algorithms use entanglement to create correlations, then use interference to extract the result.

Property	Interference	Entanglement
Qubits involved	Single qubit	Two or more qubits
Effect	Amplitudes add or cancel	Qubits become correlated
Required for speedup	Yes (essential)	Often (but not always)
Example	H-Rz-H circuit	CNOT creating Bell states

Table 6: Interference and entanglement are distinct but complementary concepts.

The Takeaway

Interference is the engine that makes quantum computing powerful.

- **Classical algorithms:** Explore one path at a time.
- **Quantum algorithms:** Explore all paths simultaneously, then use interference to select the right one.

6 Quantum Subroutines

Very roughly speaking, quantum applications tend to have the structure shown below:

Subroutine	Variable
Quantum Fourier Transform	Phase
Quantum Phase Estimation	Phase
Quantum Amplitude Amplification	Probability
Quantum Amplitude Estimation	Probability

Table 7: Quantum Subroutines covered.

6.1 Quantum Fourier Transform

The Quantum Fourier Transform (QFT) is one of the most powerful subroutines in quantum computing. It is the quantum analog of the classical discrete fourier transform (DFT), which is commonly used in signal processing, linear algebra, or differential equations.

What is a Fourier Transform?

Imagine you are listening to a song. The song is a complex mixture of sounds (guitar, vocals, drums, bass), all happening at once.

A Fourier transform is like a tool that takes that complex sound and tells you: *"This song contains these specific frequencies at these specific volumes."*

In other words:

- **Time domain:** What you hear: a waveform that changes over time.
- **Frequency domain:** What the Fourier transform gives you: a breakdown of the pure tones and patterns that make up the sound.

The Discrete Fourier Transform (DFT) does the same thing, but with discrete data points instead of continuous signals.

But crucially for quantum computing, the Fourier transform is used to find **periods** and **frequencies** in data.

The Key Insight

A Fourier transform takes data and tells you its **periodic structure**.

If you have a function that repeats every r steps, the Fourier transform will show a strong peak at the frequency $1/r$.

From Classical to Quantum

The Quantum Fourier Transform is essentially an $N \times N$ unitary matrix-vector multiplication. For a basis state $|x\rangle$, it is defined as:

$$\mathrm{QFT}\,|x\rangle = \frac{1}{\sqrt{N}} \sum_{k=0}^{N-1} \omega^{xk}\,|k\rangle\,, \quad \text{where } \omega = e^{2\pi i/N} \text{ and } N = 2^n,$$

with n being the number of qubits. This transforms the computational basis into the Fourier (phase) basis.

To implement this efficiently on a quantum computer, we must factorize this massive summation into a sequence of operations on individual qubits.

6.1.1 Binary Forms and Factorization

All the k's in the equation need to be converted to binary numbers k_j

- 1) $k = \sum_{j=1}^{n} k_j 2^{n-j}$ (Index to binary)
- 2) $|k\rangle = \bigotimes_{j=1}^{n} |k\rangle$ (Tensor Products)
- 3) $\sum_{k=0}^{2^n-1} = \sum_{k_1} ... \sum k_n$ (Multi-Index Sums)

Substituting 1) in ω^{xk}, each matrix element becomes a product series:

$\omega^{xk} = \omega^{x \sum_{j=1}^{n} k_j 2^{n-j}} = \prod_{j=1}^{n} \omega^{x k_j 2^{n-j}}$

After substituting those 3 in QFT, interchange them by factorization.

$= \bigotimes_{j=1}^{n} (|0\rangle + \omega^{x 2^{n-j}} |1\rangle)$, where $\omega^{x 2^{n-j}} = e^{2\pi i x 2^{-j}}$

Note that $e^{2\pi i x} = 1$. If not for the presence of 2^{-j}, the tensor series would have been trivial, with each term being $|0\rangle + |1\rangle$.

6.1.2 Integer Truncation

The phase applied to each state is dictated by $e^{2\pi i (x 2^{-j})}$. Because x is an integer and we are dividing it by 2^j, the result will likely have an integer part and a fractional (decimal) part.

In the complex plane, a rotation of 2π is a complete circle. Therefore, $e^{2\pi i \cdot (\text{integer})} = 1$. We can completely discard the integer part and only calculate the fractional remainder.

$\sum_{r=1}^{n} = \sum_{r=1}^{n-j} + \sum_{r=n-j+1}^{n} =$ integer + decimal.

6.1.3 Phase Expansion Example

For a three-qubit register, $x = x_1 x_2 x_3$, the phase applied to each output qubit expands naturally as follows.

QFT Equation Expansion

For j = 1 ...

$$\frac{1}{\sqrt{2}}\left(|0\rangle + e^{2\pi i x_3 2^{-1}}|1\rangle\right)$$

For j = 2 ...

$$\frac{1}{\sqrt{2}}\left(|0\rangle + \left(e^{2\pi i x_2 2^{-1}} \cdot e^{2\pi i x_3 2^{-2}}\right)|1\rangle\right)$$

$$\frac{1}{\sqrt{2}}\left(|0\rangle + e^{2\pi i\left(\frac{x_2}{2}+\frac{x_3}{4}\right)}|1\rangle\right)$$

For j = 3 ...

$$\frac{1}{\sqrt{2}}\left(|0\rangle + \left(e^{2\pi i x_1 2^{-1}} \cdot e^{2\pi i x_2 2^{-2}} \cdot e^{2\pi i x_3 2^{-3}}\right)|1\rangle\right)$$

$$\frac{1}{\sqrt{2}}\left(|0\rangle + e^{2\pi i\left(\frac{x_1}{2}+\frac{x_2}{4}+\frac{x_3}{8}\right)}|1\rangle\right)$$

Now connecting with the tensor product ...

$$|\text{Final}\rangle = \underbrace{\frac{1}{\sqrt{2}}\left(|0\rangle + e^{2\pi i(0.x_1x_2x_3)}|1\rangle\right)}_{\text{Qubit } j=3} \otimes \underbrace{\frac{1}{\sqrt{2}}\left(|0\rangle + e^{2\pi i(0.x_2x_3)}|1\rangle\right)}_{\text{Qubit } j=2} \otimes \underbrace{\frac{1}{\sqrt{2}}\left(|0\rangle + e^{2\pi i(0.x_3)}|1\rangle\right)}_{\text{Qubit } j=1}$$

6.1.4 The Final Equations

The following expression is useful when comparing the QFT with Quantum Phase Estimation, since it emphasizes the phase applied to each individual qubit.

$$QFT(|x\rangle) = \bigotimes_{j=1}^{n} \frac{1}{\sqrt{2}}\left(|0\rangle + e^{2\pi i x 2^{-j}}|1\rangle\right).$$

The next form makes the contribution of each input bit explicit and closely matches the controlled phase gates appearing in the QFT circuit.

$$QFT(|x\rangle) = \bigotimes_{j=1}^{n} \frac{1}{\sqrt{2}}\left(|0\rangle + \prod_{r=n-j+1}^{n} e^{2\pi i x_r 2^{\,n-r-j}}|1\rangle\right).$$

Many textbooks replace the explicit summation of powers of two with the more compact binary-fraction notation

$$0.x_1x_2x_3 = \frac{x_1}{2} + \frac{x_2}{4} + \frac{x_3}{8}.$$

Using this shorthand, the QFT can be written as

$$QFT(|x\rangle) = \bigotimes_{j=1}^{n} \frac{1}{\sqrt{2}}\left(|0\rangle + e^{2\pi i(0.x_{n-j+1}\cdots x_n)}|1\rangle\right).$$

Although this notation is more compact, it is mathematically equivalent to the previous expressions. The three boxed equations describe exactly the same Quantum Fourier Transform; they simply emphasize different perspectives.

What Makes the QFT Special?

The classical DFT requires $O(N^2)$ operations (or $O(N \log N)$ with the Fast Fourier Transform).

The Quantum Fourier Transform requires only $O(\log^2 N)$ operations—an exponential speedup.

Transform	**Operations**	**For $N = 2^{100}$**
Classical DFT	$O(N^2)$	2^{200} operations
Fast Fourier Transform (FFT)	$O(N \log N)$	$\approx 2^{107}$ operations
Quantum Fourier Transform (QFT)	$O(\log^2 N)$	$\approx 10^4$ operations

Table 8: The QFT provides an exponential speedup over classical Fourier transforms.

However, there is a catch: the QFT gives you the Fourier-transformed state as a superposition. To read out the result, you still need to measure it. A single measurement gives you only one random frequency component.

The Big Picture

The Quantum Fourier Transform is a tool that:

1. Takes a quantum state with hidden periodic structure.
2. Transforms it so that the periods become amplitudes in the computational basis.
3. Allows us to measure and read out those periods.

This seemingly simple operation is the key to factoring numbers (Shor's algorithm) and simulating quantum systems.

The Quantum Fourier Transform is a fast, quantum version of the classical Fourier transform. It reveals the periodic structure hidden in quantum states, and it is exponentially faster than its classical counterpart.

6.2 Quantum Phase Estimation

Quantum Phase Estimation (QPE) is one of the most powerful subroutines in quantum computing.

Recall from Section 4.3 that if a unitary operator U has an eigenstate $|v\rangle$ with eigenvalue $e^{i\phi}$:

$$U|v\rangle = e^{i\phi}|v\rangle$$

The phase ϕ is a number between 0 and 2π.

Phase Estimation answers the question: *"Given a unitary U and an eigenstate $|v\rangle$, what is the phase ϕ?"*

Why Does This Matter?

Many hard problems can be rephrased as finding the phase of a unitary operator:

- **Factoring:** Finding the period of a modular exponential function reduces to phase estimation.
- **Quantum simulation:** The energy levels of a molecule are phases of its time-evolution operator.
- **Search:** Grover's algorithm can be seen as phase estimation on a specific operator.

If you can estimate phases accurately, you can solve many important problems.

Defining the Problem

We start with:

$$U|\psi\rangle = e^{2\pi i\phi}$$

where U is a unitary matrix, and $|\psi\rangle$ is an eigenvector.

For ordinary matrices, eigenvalues can be any number. However, because U is unitary, all of its eigenvalues lie on the unit circle in the complex plane. Therefore every eigenvalue λ can be written as:

$\lambda = e^{2\pi i\phi}$, where $0 \leq \phi < 1$

Everything in Quantum Phase Estimation is designed around extracting this hidden number ϕ.

QPE uses two registers:

- an **phase register** with n qubits, which will store the binary approximation of ϕ;
- a **state register**, initialized to the eigenstate $|\psi\rangle$.

6.2.1 Step 1: Create a Superposition

Start the phase register in $|0\rangle^{\otimes n}$. Applying a Hadamard gate to each phase qubit gives an equal superposition over all possible n-bit values:

$$\frac{1}{\sqrt{2^n}} \sum_{k=0}^{2^n-1} |k\rangle|\psi\rangle$$

Instead of holding one integer k, the register now contains every possible integer simultaneously.

6.2.2 Step 2: Apply Controlled Powers of U

Each phase qubit controls a different power of U. For $n = 3$, these powers are usually arranged as

- top qubit: controls $U^{2^2} = U^4$;
- middle qubit: controls $U^{2^1} = U^2$;
- bottom qubit: controls $U^{2^0} = U$.

Because $|\psi\rangle$ is an eigenstate of U, applying U^n only adds phase:

$$U^n|\psi\rangle = e^{2\pi i\phi 2^n}|\psi\rangle$$

This is the important phase kickback step. The state register remains $|\psi\rangle$, while the estimation register picks up phases that depend on θ.

For $n = 3$, the estimation register becomes

$$|\Psi\rangle = \frac{1}{\sqrt{8}}\left(|0\rangle + e^{2\pi i(4\theta)}|1\rangle\right) \otimes \left(|0\rangle + e^{2\pi i(2\theta)}|1\rangle\right) \otimes \left(|0\rangle + e^{2\pi i\theta}|1\rangle\right)$$

Equivalently, after the controlled operations, the joint state can be written as

$$\frac{1}{\sqrt{2^n}} \sum_{k=0}^{2^n-1} e^{2\pi ik\phi}|k\rangle \otimes |\psi\rangle$$

Unfortunately, if we measured now, every basis state would still appear with equal probability because only the phases differ.

We therefore need one more transformation that converts these phase differences into measurable amplitudes.

6.2.3 Step 3: Use the Inverse QFT

Because this resulting state $|\Psi\rangle$ is mathematically identical to the output of a standard Quantum Fourier Transform, we can isolate ϕ by applying the Inverse QFT ($\mathcal{QFT}^{-1}$).

Intuitively, the inverse QFT acts like a prism separating different frequencies of light. Before the prism, all frequencies are mixed together; afterward, each frequency appears at a different location. Similarly, the inverse QFT takes the hidden phase pattern across the superposition and concentrates its probability around the computational basis state corresponding to the encoded phase.

6.2.4 Step 4: Measure and Recover the Eigenvalue

Finally, measure the phase register. The result is an integer x, written in binary. This gives the estimate

$$\theta \approx \frac{x}{2^n}$$

, where

- θ is the true fractional phase of the eigenvalue (a value between 0 and 1).
- x is the integer row number of the state vector (ranging from 0 to $2^n - 1$).
- n is the number of qubits in your estimation register.

This is the exact relationship between the row number x and the phase θ

Then the corresponding eigenvalue is

$$\lambda \approx e^{2\pi i\left(\frac{x}{2^n}\right)}$$

So QPE turns the hidden eigenvalue phase into a binary number that can be read from a measurement. Increasing the number of phase qubits improves the precision of the estimate because it allows finer binary fractions.

Although QPE may seem specialized, it is a foundational primitive in quantum algorithms because many problems can be reformulated as finding eigenvalues of suitable operators. Once those eigenvalues are expressed as phases of a unitary matrix, QPE provides an efficient way to estimate them.

6.3 Grover's Algorithm

Suppose you are given a large arbitrary list of N items. Among these items, there is one special item we want to locate; we call this the winner w.

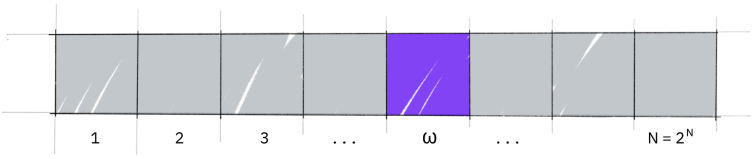


Figure 15: An unstructured list of N items.

To find w using classical computation, one would have to check on average $N/2$ items, and in the worst case, N items. On a quantum computer however, we can find the special item in roughly $\sqrt{N}$ steps with Grover's algorithm, providing a quadratic speedup for the **unstructured search problem**.

6.3.1 Step 1: State Preparation

To search on a quantum computer, we first need to store the search space, and all its possible cases into a quantum computer.

We map every item to a computational basis state:

$$x_0 \mapsto |0\rangle\,, \quad x_1 \mapsto |1\rangle\,, \quad \ldots, \quad x_j \mapsto |j\rangle\,.$$

So the basis state $|j\rangle$ represents the j-th item in the database.

Instead of checking items one by one, we apply Hadamard gates to create an equal superposition of all N states:

$$|s\rangle = \frac{1}{\sqrt{N}} \sum_{j=0}^{N-1} |j\rangle\,.$$

Every item starts with the exact same probability amplitude $\frac{1}{\sqrt{N}}$. If we measured the system right now, we would just get a random item.

6.3.2 Step 2: Oracle

We pass our superposition into a phase oracle. The oracle identifies the winner w and flips its sign (multiplies its amplitude by -1), leaving all other items alone:

$$O_f|j\rangle = (-1)^{f(j)}|j\rangle$$

where f is a function such that:

$$f(x) = \begin{cases} 1 & \text{if } j = w \\ 0 & \text{otherwise} \end{cases}$$

This function just tells us whether some x_j in our database is indeed the item that we are looking for.

After applying the oracle, the winner's amplitude is $-\frac{1}{\sqrt{N}}$, while the other $N-1$ items remain $+\frac{1}{\sqrt{N}}$. Because quantum measurement is based on the amplitude squared, measuring now would still yield a random result. We need a way to turn this phase difference into a measurable probability difference.

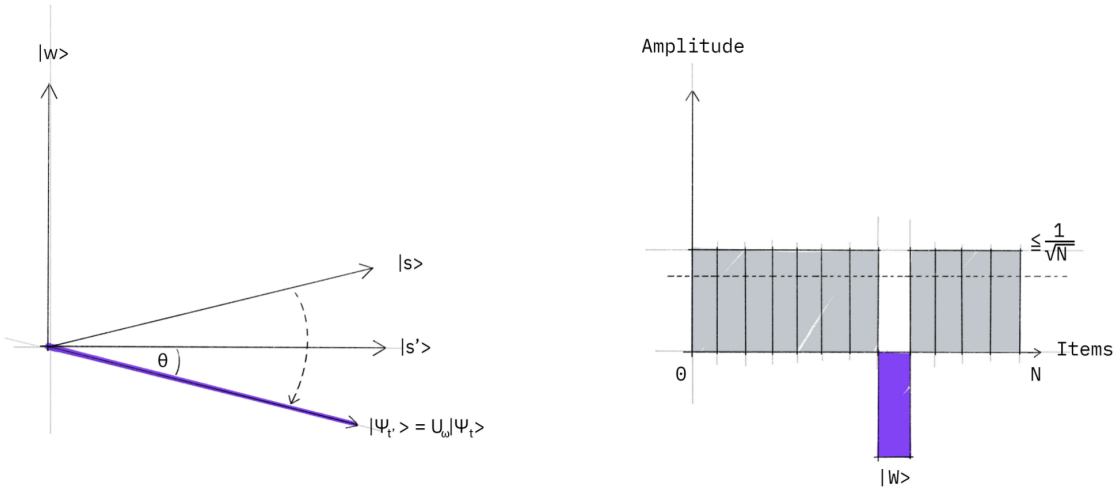


Figure 16: The winner's amplitude has been flipped to a negative value.

6.3.3 Step 3: Diffusion Operator

This step amplifies the winner's probability while shrinking the others. We reflect every amplitude across the average (mean) amplitude of the entire system. The diffusion operator is defined as:

$$D = 2\left|s\right\rangle \left\langle s\right| - I.$$

Step-by-Step Math:

1. Let μ be the average of all amplitudes. Because one item is now negative and the rest are positive, the new mean μ drops slightly below the original $\frac{1}{\sqrt{N}}$.
2. The diffusion operator applies a reflection formula to every amplitude a_j:

$$a_j^{new} = 2\mu - a_j$$

3. For the non-winners: Their amplitude a_j is slightly higher than the mean μ. When reflected across the mean, their new amplitude drops closer to zero.
4. For the winner: Its amplitude is $-\frac{1}{\sqrt{N}}$, which is far below the positive mean μ. Reflecting it across the mean shoots its amplitude way up into the positive values (roughly $\frac{3}{\sqrt{N}}$).

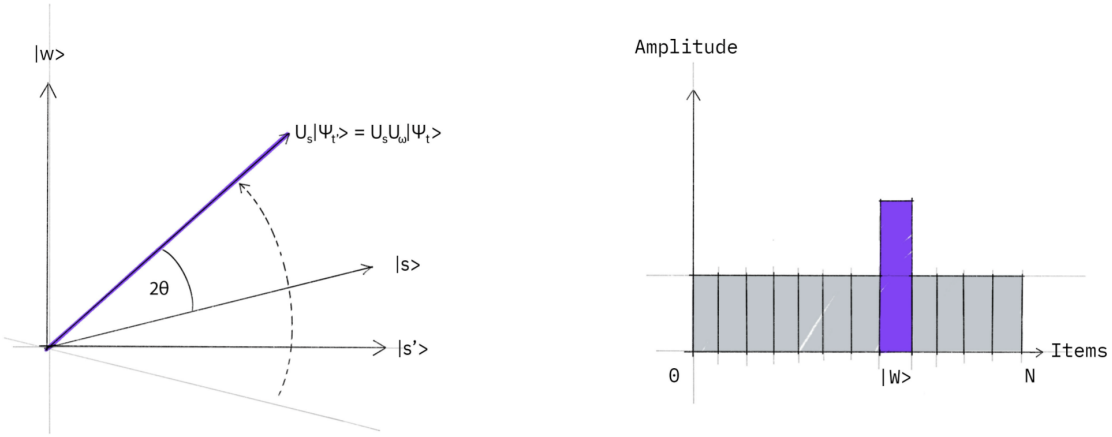


Figure 17: Amplitude distribution after the diffusion operator..

6.3.4 Step 4: Iterate

To understand how many iterations we need, we must view the process geometrically. The oracle reflects the state about the axis perpendicular to $|w\rangle$. The diffusion operator reflects the state about the axis $|s\rangle$. Their combined action, the **Grover iterate** $G = D \cdot O_f$, rotates the state by a fixed angle in the 2D plane spanned by $|w\rangle$ and the uniform superposition of all non-winners.

Let the initial angle between $|s\rangle$ and the horizontal axis be $\theta/2$, where:

$$\sin\left(\frac{\theta}{2}\right) = \frac{1}{\sqrt{N}} \quad \Longrightarrow \quad \theta \approx \frac{2}{\sqrt{N}} \quad \text{for large } N.$$

Each Grover iteration rotates the state by exactly θ. We stop rotating when the state aligns as closely as possible with $|w\rangle$, i.e., after a rotation of $\pi/2$ radians. Therefore, the optimal number of iterations is:

$$t = \left\lfloor \frac{\pi/2}{\theta} \right\rfloor \approx \frac{\pi}{4}\sqrt{N}.$$

By repeating the Oracle and Diffusion cycle approximately $\frac{\pi}{4}\sqrt{N}$ times, the amplitude of the non-winners shrinks to near zero, and the winner's amplitude approaches 1. When we measure the system now, it outputs the winner $|w\rangle$ with near certainty.

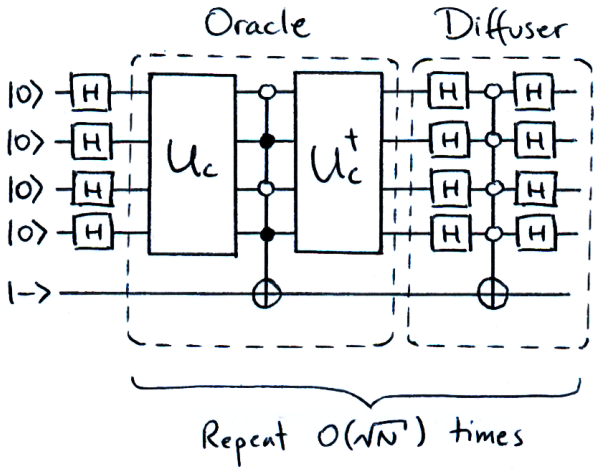


Figure 18: General Quantum Circuit for Grover's Algorithm

6.4 Quantum Amplitude Estimation

In Grover's algorithm, we amplified the probability of a marked item until it reached near 1, and then we measured it. But what if we don't want to *find* the item—what if we want to know *exactly how likely* we are to find a "good" outcome in a quantum state?

This is the goal of **Quantum Amplitude Estimation (QAE)**. It estimates the probability of measuring a "good" state without having to prepare and measure the quantum computer thousands of times.

6.4.1 The 2D Picture: Good vs. Bad

Imagine the entire quantum state existing in a simple 2D plane:

- The **x-axis** represents the "bad" states (everything we don't care about).
- The **y-axis** represents the "good" states (the outcomes we want).

Our quantum state is a unit arrow (a vector of length 1) pointing somewhere in this plane. Because it has length 1, we can describe it entirely by a single angle θ:

$$|\Psi\rangle = \cos(\theta)\ |\psi_{\text{bad}}\rangle + \sin(\theta)\ |\psi_{\text{good}}\rangle\,.$$

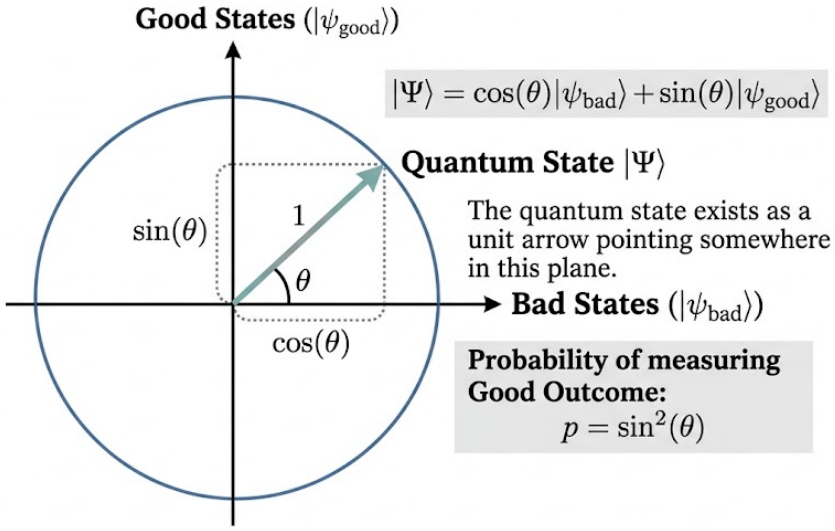


Figure 19: The quantum state as a unit vector in the 2D plane of "bad" and "good" states. The angle θ determines the probability of measuring a good outcome.

From basic trigonometry:

- The horizontal component is $\cos(\theta)$.
- The vertical component is $\sin(\theta)$.

The probability of measuring the good state is the square of the vertical component:

$$p = \sin^2(\theta).$$

If someone tells you "there is a 42% chance of measuring the good state," that is exactly equivalent to saying the arrow is pointing at a specific angle $\theta = \arcsin(\sqrt{0.42})$. Our goal in Amplitude Estimation is to figure out what θ is.

6.4.2 The Grover Operator as a "Rotator"

Recall the Grover operator (Oracle + Diffusion) from the previous section. Instead of just using it to amplify the winner until we measure it, we are going to use it as a **measuring stick**.

When we apply the Grover operator to our state, it doesn't just randomly change things—it rotates the arrow in this 2D plane by a fixed angle. Specifically, *each* application of the Grover operator rotates the state by an angle of 2θ:

$$\text{Grover}\,|\Psi\rangle \quad \Longrightarrow \quad \text{Rotate the arrow by } 2\theta.$$

Here is the crucial insight: - If θ is large (meaning the good probability is high), the arrow rotates quickly. - If θ is small (meaning the good probability is tiny), the arrow rotates very slowly.

6.4.3 The Clock Analogy (How We Measure

θ)

Imagine two clocks.

- Clock A rotates by $10°$ every second.
- Clock B rotates by $30°$ every second.

If I let each clock run for exactly 5 seconds and then look at the final position of the hands, I can easily tell which clock rotates faster. I don't need to watch them tick—I just look at where they ended up.

Amplitude Estimation uses this exact idea:

1. We apply the Grover operator a specific number of times (like letting the clock run).
2. The unknown angle θ determines *how fast* the state rotates.
3. If we can precisely measure the final angle of the arrow after many rotations, we can work backward to figure out what θ was.

6.4.4 Reading the Rotation with Quantum Phase Estimation

To measure this final rotation angle precisely, we use the **Quantum Phase Estimation (QPE)** subroutine we learned earlier.

We simply plug the Grover operator into the QPE circuit. QPE is our "super-stopwatch"—it accurately reads the phase (which is just the rotation angle) that has accumulated after many applications of the Grover operator.

Once QPE gives us an estimated angle $\tilde{\theta}$, we plug it directly into our trigonometry:

$$\boxed{\tilde{p} = \sin^2(\tilde{\theta})}$$

This $\tilde{p}$ is our final estimate of the probability of measuring a good state.

The Core Idea

Amplitude Estimation turns the problem of *counting* or *estimating probabilities* into a problem of *measuring rotation speeds*.

Instead of rolling a quantum die thousands of times to estimate the odds of a "6," we just run the Grover "rotator" a few times, measure the final angle with QPE, and use $\sin^2$ to get the odds in one go.

6.4.5 The Quadratic Speedup (Without the Heavy Math)

Why is this useful? - Classically, to estimate a probability p to a reasonable accuracy, you have to take roughly $1/p$ measurements. If the probability is 1%, you need to check the system about 100 times. - With Amplitude Estimation, the number of times you need to apply the Grover operator scales like $1/\sqrt{p}$. For a 1% probability, that means only about 10 applications.

This is the same quadratic speedup we saw in Grover's search, but now applied to the general problem of *estimating amplitudes*. It is the foundation for quantum advantage in fields like finance (risk analysis), physics (simulating cross-sections), and machine learning (computing inner products).

6.4.6 Bringing It Back to Grover

If we specialize Amplitude Estimation to Grover's original search problem: - The "good" state $|\psi_{\text{good}}\rangle$ is just the marked winner $|w\rangle$. - The initial probability is $p = 1/N$, which means $\theta \approx 1/\sqrt{N}$. - Running Amplitude Estimation measures this tiny rotation angle, allowing us to figure out exactly how many items N are in the database without searching through all of them!

This makes Amplitude Estimation a strictly more powerful generalization of Grover's algorithm. Grover tells us *where* the item is; Amplitude Estimation tells us *how likely* it is to appear, which is the key to counting and estimation tasks.

7 Solving Linear Systems

Solving a system of linear equations $Ax = b$ is a common and important computational problem in many fields such as engineering, mathematics, finance, and computer science.

In classical computing, this is typically solved using methods such as Gaussian elimination or conjugate gradient, taking polynomial time $O(N^3)$ for an $N \times N$ matrix. But with advances in these fields, we encounter ever larger systems of equations where even polynomial time is too costly.

This is where quantum computing can offer exponential speedups to computation time. The HHL algorithm is one such implementation.

7.1 HHL Algorithm

The Harrow–Hassidim–Lloyd (HHL) algorithm is a quantum algorithm for obtaining certain limited information about the solution to the matrix equation $A|x\rangle = |b\rangle$.

The algorithm requires the following assumptions to hold:

1. The matrix A must be Hermitian so that it can be exponentiated into a unitary operator via Hamiltonian simulation.
2. The algorithm requires an efficient procedure to prepare $|b\rangle$ as a quantum state.
3. The system is sparse, has a low condition number κ, and the user is only interested in a sample of the solution (not the full vector).

Hermitian Matrix

A square matrix that is equal to its conjugate transpose matrix ($A = A^\dagger$). Diagonal entries are strictly real numbers.

$$\begin{bmatrix} 4 & 6+i \\ 6-i & 2 \end{bmatrix} \xrightarrow{\text{Conjugate}} \begin{bmatrix} 4 & 6-i \\ 6+i & 2 \end{bmatrix} \xrightarrow{\text{Transpose}} \begin{bmatrix} 4 & 6+i \\ 6-i & 2 \end{bmatrix}$$

The first and last matrix are equal.

7.2 Initial Setup

We start with a linear system problem

$$Ax = b$$

where:

- A is a known $N \times N$ Hermitian matrix.
- b is a known N-dimensional vector.
- x is the unknown vector we want to find.

A classical algorithm would directly compute:

$$x = A^{-1}b$$

The HHL algorithm instead prepares a quantum state $|x\rangle$ whose amplitudes correspond to the entries of the solution vector.

HHL uses three main registers:

1. **State register:** Stores $|b\rangle$ and eventually becomes proportional to $|x\rangle$.
2. **Clock register:** Stores the phase/eigenvalue estimate produced by QPE. The number of qubits in this register controls the precision of the eigenvalue estimate.
3. **Ancilla qubit:** The least significant bit, used for the controlled rotation that applies the factor $1/\lambda_k$.

7.3 Eigendecomposition

Suppose A has eigenvectors $v_0, v_1, \ldots, v_{N-1}$ with corresponding eigenvalues $\lambda_0, \lambda_1, \ldots, \lambda_{N-1}$. Then by definition:

$$Av_k = \lambda_k v_k$$

Collecting all eigenvectors into a matrix V and all eigenvalues into a diagonal matrix Λ gives:

$$AV = V\Lambda$$

If V^{-1} exists, we can isolate A:

$$A = V\Lambda V^{-1}$$

Taking the inverse gives:

$$A^{-1} = V\Lambda^{-1}V^{-1}$$

Any vector can be written as a combination of the eigenvectors. In summation form:

$$b = \sum_{k=0}^{N-1} c_k v_k$$

The coefficients $c = V^{-1}b$ simply tell us how much of each eigenvector is present in b.

Applying A^{-1} gives:

$$x = A^{-1}b = \sum_{k=0}^{N-1} \frac{c_k}{\lambda_k} v_k$$

The Intuition

Look at the similarity between b and x!

The solution vector has the exact same eigenvector directions as b, but each component is simply multiplied by $1/\lambda_k$.

Once we recognize this, the problem becomes much more specific. The quantum computer must somehow:

1. Identify each eigencomponent v_k.
2. Determine its eigenvalue λ_k.
3. Multiply that component by $1/\lambda_k$.

7.4 Algorithm Workflow

Step 1: Preparing the input

Instead of classical decomposition, b is loaded directly into a quantum superposition:

$$|b\rangle = \sum_{k=0}^{N-1} c_k |v_k\rangle$$

Here, the eigenvectors $|v_k\rangle$ act as the basis states, and the coefficients c_k act as the probability amplitudes. Because b is now stored as a superposition of all eigenvectors simultaneously, we can focus entirely on applying the $1/\lambda_k$ transformation to find x.

Step 2: Hamiltonian Simulation

We cannot input matrix A directly into a quantum computer unless it is unitary. Unfortunately, the matrix A in our linear system is typically not unitary.

This is why we assume A is Hermitian: because Hermitian matrices can be evolved into valid quantum gates using a process called **Hamiltonian simulation**.

The key insight is that for any Hermitian matrix A, the matrix exponential:

$$U = e^{iAt}$$

is unitary. This is known as the **time evolution operator** or **Hamiltonian** of the system.

What is Hamiltonian Simulation?

Hamiltonian simulation is the process of constructing a quantum circuit that implements the unitary operator e^{iAt} for a given Hermitian matrix A.

Think of it like this: if A is the "energy" of a quantum system, then e^{iAt} is the operator that describes how that system evolves over time t.

The parameter t is chosen carefully. For HHL, we typically set $t = 2\pi$ so that the eigenvalues of $U = e^{iA}$ are $e^{2\pi i \lambda_k / 2^m}$, where m is the number of qubits in the clock register.

Step 3: Apply Controlled Powers

Quantum Phase Estimation cannot determine the eigenvalues from a single application of $U = e^{iAt}$. Instead, it repeatedly applies larger and larger powers of the unitary.

Specifically, the algorithm uses:

$$U, U^2, U^4, U^8, \ldots, U^{2^{m-1}}$$

where each power is controlled by one qubit in the clock register.

Why Multiple Powers?

Think of measuring a phase with a ruler.

If you only have a ruler marked in whole numbers, you can only measure to the nearest integer.

If you have rulers marked in halves, quarters, and eighths, you can measure much more precisely.

The controlled powers U^{2^k} provide precisely this "fractional" precision. The more qubits in the clock register, the finer the precision of the eigenvalue estimate.

The effect of these controlled operations is to encode the phase information from the eigenvalues λ_k into the clock register.

Step 4: Quantum Phase Estimation

We now apply Quantum Phase Estimation (QPE) to the state register using the controlled-U^{2^k} operations from Step 3.

After QPE, the state becomes:

$$\sum_{k=0}^{N-1} c_k |\tilde{\lambda}_k\rangle |v_k\rangle$$

where $|\tilde{\lambda}_k\rangle$ is an m-qubit register encoding an estimate of the eigenvalue λ_k.

More precisely, the clock register stores the binary approximation:

$$|\tilde{\lambda}_k\rangle = \left|\frac{\lambda_k}{2\pi}\right\rangle$$

which is a binary representation of the phase.

At this point:

- The **state register** is still in the eigenstate $|v_k\rangle$ (it was not disturbed).
- The **clock register** now contains the eigenvalue estimate.
- The **ancilla qubit** is still in $|0\rangle$.

Step 5: Controlled Rotation

Instead of literally performing division, HHL introduces an ancillary qubit and performs a controlled rotation whose angle depends on the eigenvalue stored in the clock register.

The rotation is chosen so that:

$$\sin(\theta_k) \propto \frac{1}{\lambda_k}$$

More precisely, the controlled rotation applies:

$$R_y(\theta_k) = \begin{pmatrix} \cos(\theta_k) & -\sin(\theta_k) \\ \sin(\theta_k) & \cos(\theta_k) \end{pmatrix}$$

where:

$$\theta_k = 2 \arcsin\left(\frac{C}{\lambda_k}\right)$$

and C is a normalization constant chosen to ensure that $\frac{C}{\lambda_k} \leq 1$ for all eigenvalues.

The Controlled Rotation Step

The controlled rotation applies a rotation to the ancilla qubit that is *conditional* on the eigenvalue λ_k stored in the clock register.

After the rotation, the state becomes:

$$\sum_{k=0}^{N-1} c_k |\tilde{\lambda}_k\rangle |v_k\rangle \left(\sqrt{1 - \frac{C^2}{\lambda_k^2}}|0\rangle + \frac{C}{\lambda_k}|1\rangle\right)$$

The amplitude of the ancilla being in state $|1\rangle$ is proportional to $1/\lambda_k$.

This cleverly avoids the need to perform actual division. The $1/\lambda_k$ factor is encoded in the *amplitude* of the ancilla qubit.

Step 6: Uncompute and Measure

We must now **measure only the ancilla qubit**.

If we measure the ancilla and it is in state $|1\rangle$, the state collapses to:

$$\sum_{k=0}^{N-1} c_k \frac{C}{\lambda_k} |\tilde{\lambda}_k\rangle |v_k\rangle |1\rangle$$

The $1/\lambda_k$ factor is now "locked into" the state register through its correlation with the ancilla. If we discard the ancilla (or simply ignore it), the state register is left in the unnormalized state:

$$|x\rangle \propto \sum_{k=0}^{N-1} \frac{c_k}{\lambda_k} |v_k\rangle$$

- If the ancilla measures as $|1\rangle$, the operation succeeded, and the $1/\lambda_k$ factor is successfully applied to our state.
- If the ancilla measures as $|0\rangle$, the operation failed, and we must discard the run and try again.

Why is This Probabilistic?

Quantum algorithms are inherently probabilistic!

The controlled rotation only *partially* rotates the ancilla. There is always some probability of measuring $|0\rangle$, which corresponds to the "failure" case.

The success probability is:

$$P_{\text{success}} = \sum_{k=0}^{N-1} \left| \frac{c_k C}{\lambda_k} \right|^2$$

This depends on the eigenvalues and the condition number of A. For well-conditioned systems, the success probability can be high.

Step 7: Readout

After a successful measurement, the state register contains:

$$|x\rangle = \frac{\sum_{k=0}^{N-1} \frac{c_k}{\lambda_k} |v_k\rangle}{\sqrt{\sum_{k=0}^{N-1} \left| \frac{c_k}{\lambda_k} \right|^2}}$$

This is the normalized quantum state whose amplitudes are proportional to the solution vector x.

What Does HHL Actually Output?

Suppose the actual solution is:

$$x = \begin{bmatrix} 4 \\ 3 \end{bmatrix}$$

HHL does *not* output the numbers 4 and 3.

Instead, it prepares the quantum state:

$$|x\rangle = \frac{4}{5}|0\rangle + \frac{3}{5}|1\rangle$$

The amplitudes encode the solution. To recover the actual vector, you would need to perform tomography (measure many copies of the state) or use the state as input to another quantum algorithm.

The number of measurements required to reconstruct x depends on the desired accuracy and the norm of x.

7.5 The Complete State Evolution

Let us trace the entire state through the algorithm:

1. **Initial:** $|0\rangle^{\otimes m} \otimes |0\rangle_a \otimes |b\rangle$
2. **After QPE:** $\sum_k c_k |\tilde{\lambda}_k\rangle \otimes |0\rangle_a \otimes |v_k\rangle$
3. **After controlled rotation:** $\sum_k c_k |\tilde{\lambda}_k\rangle \otimes \left(\sqrt{1 - \frac{C^2}{\lambda_k^2}}|0\rangle_a + \frac{C}{\lambda_k}|1\rangle_a\right) \otimes |v_k\rangle$
4. **After inverse QPE (uncompute):** $\sum_k c_k |0\rangle^{\otimes m} \otimes \left(\sqrt{1 - \frac{C^2}{\lambda_k^2}}|0\rangle_a + \frac{C}{\lambda_k}|1\rangle_a\right) \otimes |v_k\rangle$
5. **After measuring ancilla $= |1\rangle$:** $\sum_k c_k \frac{C}{\lambda_k}|v_k\rangle \propto |x\rangle$

7.6 Key Takeaways

- **Exponential speedup:** HHL solves linear systems in $O(\kappa^2 \log N)$ time, compared to classical $O(N^3)$.
- **Limited output:** HHL prepares a quantum state $|x\rangle$, not the classical vector x.
- **Condition number dependence:** The runtime scales with κ^2, so ill-conditioned systems are still hard.
- **Sparsity requirement:** HHL requires A to be sparse for efficient Hamiltonian simulation.
- **Probabilistic:** The algorithm succeeds with some probability, requiring multiple runs.

The Big Picture

HHL is a beautiful demonstration of quantum advantage. It combines:

- Hamiltonian simulation (to exponentiate A)
- Quantum Phase Estimation (to find eigenvalues)
- Controlled rotations (to apply $1/\lambda_k$)
- Interference and measurement (to extract the result)

While HHL has limitations, it shows the potential of quantum computers to solve problems that are intractable classically.

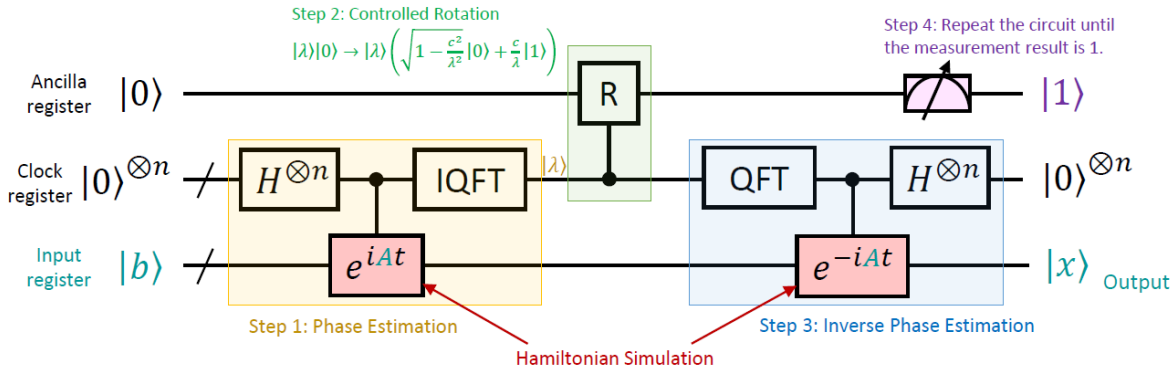


Figure 20: HHL Circuit.

8 Shor's Algorithm

8.1 Motivation

Math time! Here's a challenge: try to find the prime factors of 15. It was 3 and 5, right? Pretty simple. A classical computer can do this without breaking a sweat.

What about the factors of 216? Takes a little more time, but till trivial - $216 = 2^3 \cdot 3^3$.

What about the prime factors of

```
1230186684530117755130494958384962720772853569595334792197322452151726400507263657518745202199786469389956474942774063845925192557326303453731548268507917026122142913461670429214311602221240479274737940806653514195974598569021434 13
```

That's a 232-digit number. Multiplying two primes of that size is a breeze for a computer, but working backwards to find those primes? That is ridiculously hard because there are just so many possibilities. In fact, with the best classical algorithms known, factoring a number of this size takes hundreds of years of computing time. Basically, **factoring is hard**.

And that fact is the backbone of RSA, one of the most widely-used public-key encryption methods that protects your bank transactions, private emails, and digital signatures.

RSA works by choosing a large composite number N (the product of two secret prime numbers) and using it to encrypt or 'lock' a message. Anyone can encrypt if they know N, but decrypting or 'unlocking' the data without knowing the prime factors is hopeless without investing lots of time and computing resources — unless you can factor N quickly.

But alright, let's say you want to break this RSA encryption, in order to rob some banks, read your ex's email, whatever. What this means is that, if we want a fast factoring algorithm, we are going to have to exploit some deep mathematical structure in the factoring problem. Fortunately, factoring has a special property we can use: it can be reduced to finding the period of a repeating sequence.

8.2 The Mathematics of Factoring

Before we dive into quantum mechanics, we need to understand the mathematical engine that makes Shor's algorithm tick. The entire algorithm rests on a beautiful reduction: if we can find the period of a certain repeating sequence, we can extract the factors of N.

Lets look at this particular sequence: the powers of 2.

$$2,\ 4,\ 8,\ 16,\ 32,\ 64,\ 128,\ 256,\ 512,\ 1024,\ \ldots$$

Now, let us look at the powers of 2 *modulo* 15: in other words, the remainder left when we divide each power of 2 by 15:

$$2,\ 4,\ 8,\ 1,\ 2,\ 4,\ 8,\ 1,\ 2,\ 4,\ \ldots$$

As you can see, taking the powers of 2 mod 15 gives us a **periodic sequence**, whose period (i.e., how far you have to go before it starts repeating) is 4.

Euler's Totient Theorem

Let $N = p \cdot q$ be a product of two distinct primes, and let a be any integer not divisible by p or q. Consider the sequence

$$a^1 \bmod N,\ a^2 \bmod N,\ a^3 \bmod N,\ a^4 \bmod N,\ \ldots$$

Then this sequence is periodic, and its period r (the smallest positive integer such that $a^r \equiv 1 \pmod{N}$) always divides $(p-1)(q-1)$.

For example, with $N = 15$, we have $p = 3$, $q = 5$, so $(p-1)(q-1) = 2 \cdot 4 = 8$. Indeed, the period of the powers of 2 was 4, which divides 8. If we had picked $a = 7$ instead, the period would be 4 as well. For $N = 21$ ($p = 3, q = 7$), we have $(p-1)(q-1) = 12$, and the period of 2 modulo 21 is 6 — again, a divisor of 12.

Now, I will admit: knowing the period r is not the same as knowing p and q themselves. But grant me that it is *something*. In fact, it is more than something. It turns out that if we could learn the periods r_a for several randomly chosen values of a, then with high probability we could piece together enough information to reconstruct $(p-1)(q-1)$ exactly. And once we knew (p-1)(q-1), we could then use a few mathematical tricks to recover p and q, the prime factors we wanted.

So what's the catch? Well, even though the sequence

$$a^1 \bmod N,\ a^2 \bmod N,\ a^3 \bmod N,\ a^4 \bmod N,\ \ldots$$

will eventually start repeating itself, the number of steps before it repeats could be almost as large as N itself — and N might have hundreds or thousands of digits! This is why finding the period doesn't seem to lead to a fast classical factoring algorithm.

But what if we had a quantum computer? Could we put

$a^1 \bmod N,\ a^2 \bmod N,\ a^3 \bmod N,\ a^4 \bmod N,\ \ldots$

... in some kind of superposition?

8.3 Shor's Algorithm: Finding the Period

Given a positive integer $N = pq$, where p and q are both prime, Shor's algorithm allows us to find p and q in roughly $O(\log^3(N))$ steps, an exponential speedup over classical limits.

Shor's genius was realizing that while classical computers must evaluate the sequence one by one, a quantum computer can evaluate the entire sequence at once and extract the period using our old friend: the **Quantum Fourier Transform (QFT)**.

8.3.1 Step 1: State Preparation

Choose a random integer a, where $1 < a < N$, and $\gcd(a, N) = 1$. (If $\gcd(a, N) > 1$, you're incredibly lucky—you just found a factor and don't even need the quantum computer!)

We need two quantum registers. Register 1 (the evaluation register) has t qubits, where $Q = 2^t$ is large enough to prevent aliasing (typically $N^2 \leq Q < 2N^2$). Register 2 (the target register) has enough qubits to store numbers up to N.

We initialize both registers to $|0\rangle$. Then, we apply Hadamard gates to every qubit in Register 1 to create an equal superposition of all possible inputs x from 0 to $Q-1$:

$$|\psi_1\rangle = \frac{1}{\sqrt{Q}} \sum_{x=0}^{Q-1} |x\rangle\,|0\rangle$$

8.3.2 Step 2: Modular Exponentiation (The Oracle)

Next, we apply a quantum circuit that computes our sequence: $f(x) = a^x \bmod N$. It evaluates this function for every x simultaneously and stores the output in Register 2, using Register 1 as the control.

This creates an **entangled state** between the inputs and their corresponding sequence values:

$$|\psi_2\rangle = \frac{1}{\sqrt{Q}} \sum_{x=0}^{Q-1} |x\rangle \, |a^x \bmod N\rangle$$

While this state theoretically holds the entire sequence, measuring it directly won't give us the period. Measurement collapses the state, yielding just one random pair of $(x, a^x \bmod N)$.

Figure 21: Register 1 holds the inputs while Register 2 holds the repeating sequence.

8.3.3 Step 3: Conceptual Measurement

To understand how we extract the period r, imagine we measure Register 2 right now. It will collapse to some random value k from our sequence (for example, if $N = 15$ and $a = 2$, k might be 8).

Because the registers are entangled, Register 1 immediately collapses into a superposition of *only* the x values that produced k. Since the sequence is periodic with period r, these x values are spaced exactly r apart (e.g., $x_0, x_0 + r, x_0 + 2r, \dots$).

Register 1 is now in a new state:

$$|\psi_3\rangle \approx \frac{1}{\sqrt{m}} \sum_{j=0}^{m-1} |x_0 + jr\rangle$$

where m is the number of times the period fits into Q. We have successfully encoded the periodicity r into the quantum amplitudes of Register 1! (Note: In practice, we don't actually need to measure Register 2; the math works out exactly the same via partial trace).

8.3.4 Step 4: The Quantum Fourier Transform (QFT)

We still cannot measure Register 1 directly, as it would just yield a random $x_0 + jr$, giving us no information about r. We need to transform the state so that the periodicity r becomes measurable.

We apply the Quantum Fourier Transform (QFT) to Register 1. The QFT maps a computational basis state $|x\rangle$ to a superposition of states with rotating complex phases:

$$\mathrm{QFT}\,|x\rangle = \frac{1}{\sqrt{Q}} \sum_{y=0}^{Q-1} e^{2\pi i \frac{xy}{Q}} \, |y\rangle$$

When we apply this to our periodic state $|\psi_3\rangle$, something beautiful happens with the complex amplitudes:

1. **Destructive Interference:** For most target states $|y\rangle$, the complex phases $e^{2\pi i \cdots}$ point in all different directions around the unit circle. When added together, they sum to zero. The probability of measuring these states vanishes.
2. **Constructive Interference:** However, when y is a multiple of Q/r (i.e., $y \approx c\frac{Q}{r}$ for some integer c), all the complex phases align in the exact same direction. Their amplitudes stack, creating massive probability spikes.

Figure 22: The QFT creates constructive interference spikes at multiples of Q/r.

8.3.5 Step 5: Measurement and Continued Fractions

We finally measure Register 1. Thanks to the QFT, with very high probability we will measure a state y that is close to a multiple of Q/r. We can rewrite this algebraically as:

$$\frac{y}{Q} \approx \frac{c}{r}$$

Because we just measured y, and we built the computer so we already know Q, the left side of this equation is a known decimal.

We hand this decimal back to a classical computer and use an algorithm called **Continued Fractions**, which finds the closest fraction to our decimal. The denominator of that fraction is our period, r!

8.4 Extracting the Prime Factors

Now that the quantum computer has done the heavy lifting and given us the order r, we use classical math to finish the job.

If r is an even number (and $a^{r/2} \not\equiv -1 \pmod{N}$), we can do some simple algebra:

$$a^r - 1 = (a^{r/2} - 1)(a^{r/2} + 1) \equiv 0 \pmod{N}$$

This means N evenly divides the product $(a^{r/2}-1)(a^{r/2}+1)$. However, N does *not* divide either piece individually. Therefore, each piece must share a nontrivial factor with N.

We simply compute the Greatest Common Divisor (GCD) using the classical Euclidean algorithm:

$$\gcd(a^{r/2} - 1, N) \quad \text{and} \quad \gcd(a^{r/2} + 1, N)$$

And voilà—these computations output p and q, cracking the RSA encryption.

8.5 Current Status

While mathematically sound, Shor's algorithm is a massive engineering challenge. To factor a 2048-bit RSA key, we would need a fault-tolerant quantum computer with millions of physical qubits to manage error rates and decoherence.

Currently, quantum computers have only successfully factored very small numbers (like 15 and 21) using simplified, compiled versions of Shor's algorithm. Breaking real-world cryptography remains decades away, but the theoretical threat is severe enough that the cybersecurity world is actively migrating to "Post-Quantum Cryptography" standards today.

9 References

This bibliography lists the main outside resources used while preparing these notes. The references are included as general sources and image sources, rather than as in-text citations.

1. Eric R. Johnston, Nic Harrigan, and Mercedes Gimeno-Segovia. *Programming Quantum Computers: Essential Algorithms and Code Samples*. O'Reilly Media, Inc., 2019. ISBN: 9781492039655.
2. Scott Aaronson. "Shor, I'll do it." *Shtetl-Optimized* (blog), February 24, 2007. https://scottaaronson.blog/?p=208.
3. cnot.io. "Grover's Algorithm." *cnot.io: Quantum Computing Explained*. https://cnot.io/quantum_algorithms/grover/grovers_algorithm.html.

A Mathematical Foundations

This appendix provides a brief refresher on the mathematical tools used throughout this text. For a full treatment, the reader is encouraged to consult standard linear algebra and complex analysis textbooks.

A.1 Complex Numbers

Quantum states are described using complex numbers. A complex number is defined as:

$$z = a + ib,$$

where a and b are real numbers, and i is the imaginary unit satisfying $i^2 = -1$.

Key operations:

- **Complex conjugate**: $z^* = a - ib$. This flips the sign of the imaginary part.
- **Modulus (magnitude)**: $|z| = \sqrt{a^2 + b^2} = \sqrt{zz^*}$. This represents the "length" of the complex number.
- **Phase**: Any complex number can be written in polar form using **Euler's formula**:

  $$e^{i\theta} = \cos\theta + i\sin\theta.$$

  This is extremely important in quantum computing because global and relative phases (the θ in $e^{i\theta}$) affect how quantum states interfere with each other.

A.2 Linear Algebra Basics

Quantum states are represented as vectors in a complex vector space (Hilbert space).

- **State vectors (Kets)**: A quantum state is written as a column vector $|v\rangle$. For example, a single qubit state is $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$.
- **Bras and inner products**: The conjugate transpose of $|v\rangle$ is the row vector $\langle v|$. The inner product between two states $\langle u|v\rangle$ is a complex number that measures their "overlap."
- **Norm**: The length of a state vector is $\sqrt{\langle v|v\rangle}$. All valid quantum states have unit norm ($\langle v|v\rangle = 1$), ensuring probabilities sum to 1.
- **Orthogonality and bases**: Two states are orthogonal if $\langle u|v\rangle = 0$. A set of orthonormal basis vectors (e.g., $\{|0\rangle, |1\rangle\}$) spans the entire space, allowing us to express any vector as a linear combination.
- **Matrices and operators**: Quantum gates are represented by matrices. Matrix multiplication corresponds to applying sequential gates to a state. A matrix U is **unitary** if $U^\dagger U = UU^\dagger = I$, where $U^\dagger$ is the conjugate transpose. Unitary matrices preserve the norm of a vector, which is why all quantum gates must be unitary.

A.3 The Tensor Product ($\otimes$)

To describe systems with multiple qubits, we combine their vector spaces using the tensor product. If $|u\rangle$ is an m-dimensional vector and $|v\rangle$ is an n-dimensional vector, their tensor product $|u\rangle \otimes |v\rangle$ is an $m \times n$-dimensional vector.

For two qubits:

$$|0\rangle \otimes |0\rangle = |00\rangle = \begin{pmatrix} 1 \\ 0 \\ 0 \\ 0 \end{pmatrix}, \quad |0\rangle \otimes |1\rangle = |01\rangle = \begin{pmatrix} 0 \\ 1 \\ 0 \\ 0 \end{pmatrix}, \quad \text{etc.}$$

In general, for arbitrary states:

$$\begin{pmatrix} a \\ b \end{pmatrix} \otimes \begin{pmatrix} c \\ d \end{pmatrix} = \begin{pmatrix} ac \\ ad \\ bc \\ bd \end{pmatrix}.$$

The tensor product is the mathematical foundation for multi-qubit systems and entanglement, as it allows us to construct the combined Hilbert space from individual qubit spaces.

A.4 Eigenvalues and Eigenvectors

A central concept in linear algebra and quantum mechanics is the eigenvalue equation. For a matrix A and a non-zero vector $|v\rangle$:

$$A|v\rangle = \lambda|v\rangle,$$

where λ is a scalar called the **eigenvalue**, and $|v\rangle$ is the corresponding **eigenvector**.

Why this matters for quantum computing:

- **Measurement**: The possible outcomes of measuring an observable are precisely its eigenvalues. For Hermitian matrices ($A = A^\dagger$), these eigenvalues are always real numbers.
- **Quantum gates**: The eigenvectors of a unitary gate (like the Pauli gates or the Grover operator) define the axes of rotation in the Bloch sphere or higher-dimensional spaces.
- **Quantum Phase Estimation (QPE)**: The entire QPE algorithm is built around extracting the eigenvalues of a unitary operator. By encoding a problem's solution into the eigenvalues of a matrix, we can read out the answer via phase estimation.